LINUX COMMAND REFERENCE AND PRACTICAL LAB GUIDE

(Chapters 1–4)

Coursework Project

Name: Mohamed Adan Mohamud

ID: C6240044

Class: CNS241

Semester: 5

Subject: Linux System Administration

Date: 10 November 2026

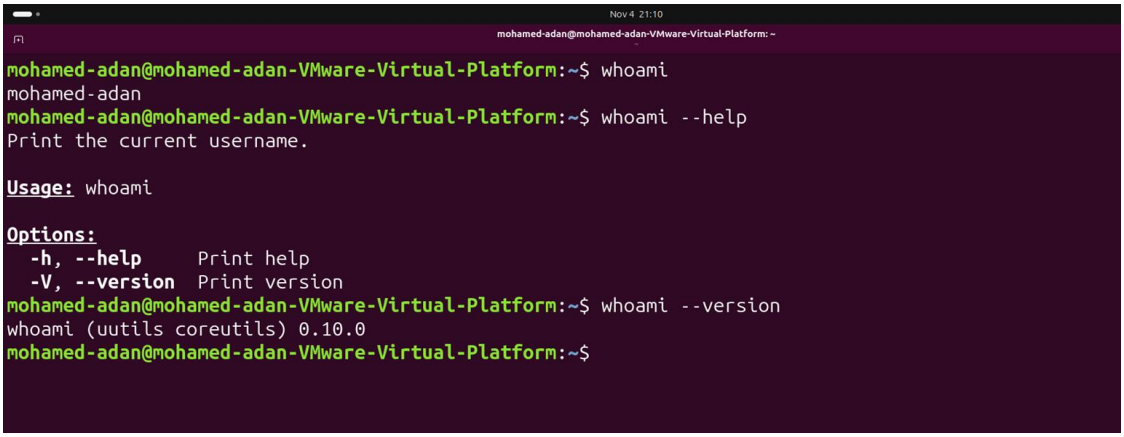
1. The whoami Command

The **whoami** command is used to display the username of the user currently logged into the system. It is a quick way to verify your current identity and check which account privileges you are operating under.

Options:

This **whoami** command is very simple and generally does not require any options. However, it accepts standard help flags:

- **--help:** Displays a help message and exits.
- **--version:** Outputs version information and exits.

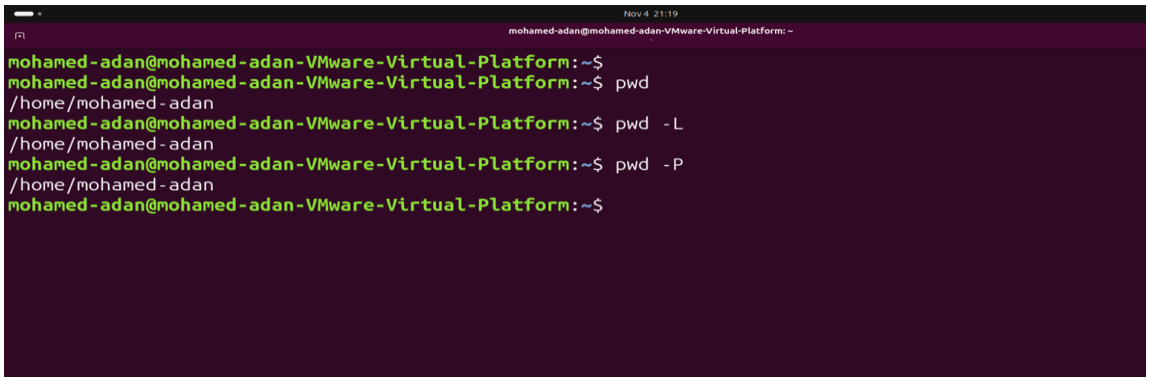


2. The pwd Command

pwd stands for **Print Working Directory.** It outputs the absolute path of the directory you are currently working in. This is essential for navigating the Linux file system without getting lost.

Options:

- **-L:** (Logical) Prints the path using the environment variable $PWD, even if it contains symbolic links. (This is the default behavior).
- **-P:** (Physical) Prints the actual physical path, resolving any symbolic links to their true directory locations.

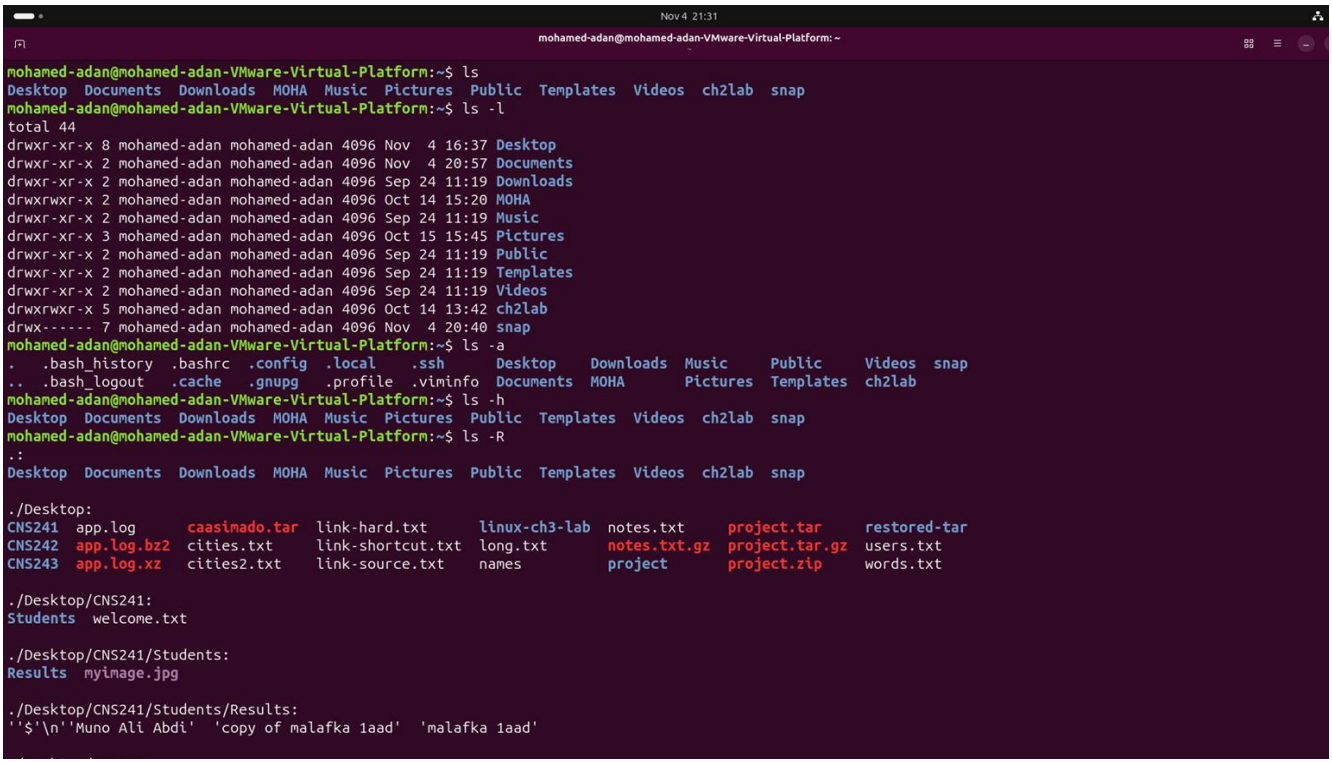


3. The ls Command

ls stands for **List**. It is one of the most frequently used commands. It displays the files and directories contained within the current working directory.

Options:

- **-l:** Uses a "long listing" format. It shows file permissions, number of links, owner, group, file size, and the date/time the file was last modified.
- **-a:** (All) Lists all files, including hidden files (files that start with a dot .).
- **-h:** (Human-readable) Used with -l to print file sizes in a readable format (e.g., 1K, 234M, 2G) instead of raw bytes.
- **-R:** (Recursive) Lists the contents of directories, and the contents of their subdirectories.

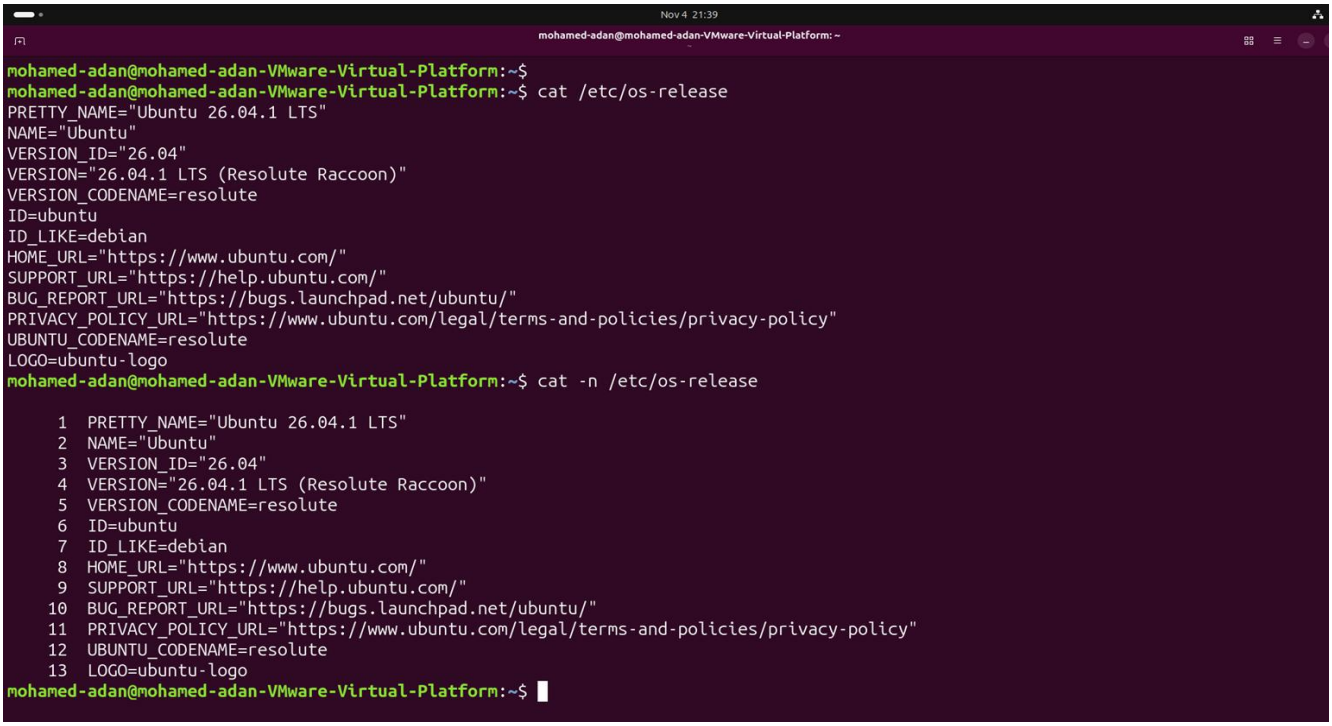


4. The cat /etc/os-release Command

The **cat (concatenate)** command reads files and prints their contents to the terminal. In this specific exercise, we are using it to read the /etc/os-release file. This file contains the operating system identification data, telling you the exact version of Ubuntu you are running.

Options (for cat):

- **-n:** Numbers all output lines.
- **-b:** Numbers non-blank output lines.
- **-E:** Displays a $ at the end of each line to show where the line breaks are.

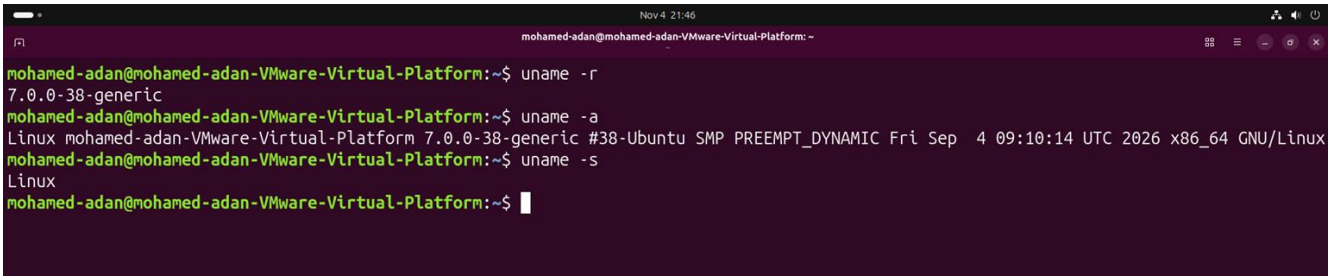


5. The uname -r Command

uname stands for **Unix Name.** It prints system information about the machine and the operating system. When used with the -r flag, it specifically prints the kernel release version. This tells you the core version of the Linux system running underneath the Ubuntu desktop.

Options:

- **-r:** Prints the kernel release version.
- **-a:** Prints all available system information (kernel name, hostname, kernel release, version, machine hardware name, processor type, etc.).
- **-s:** Prints the kernel name (e.g., Linux).

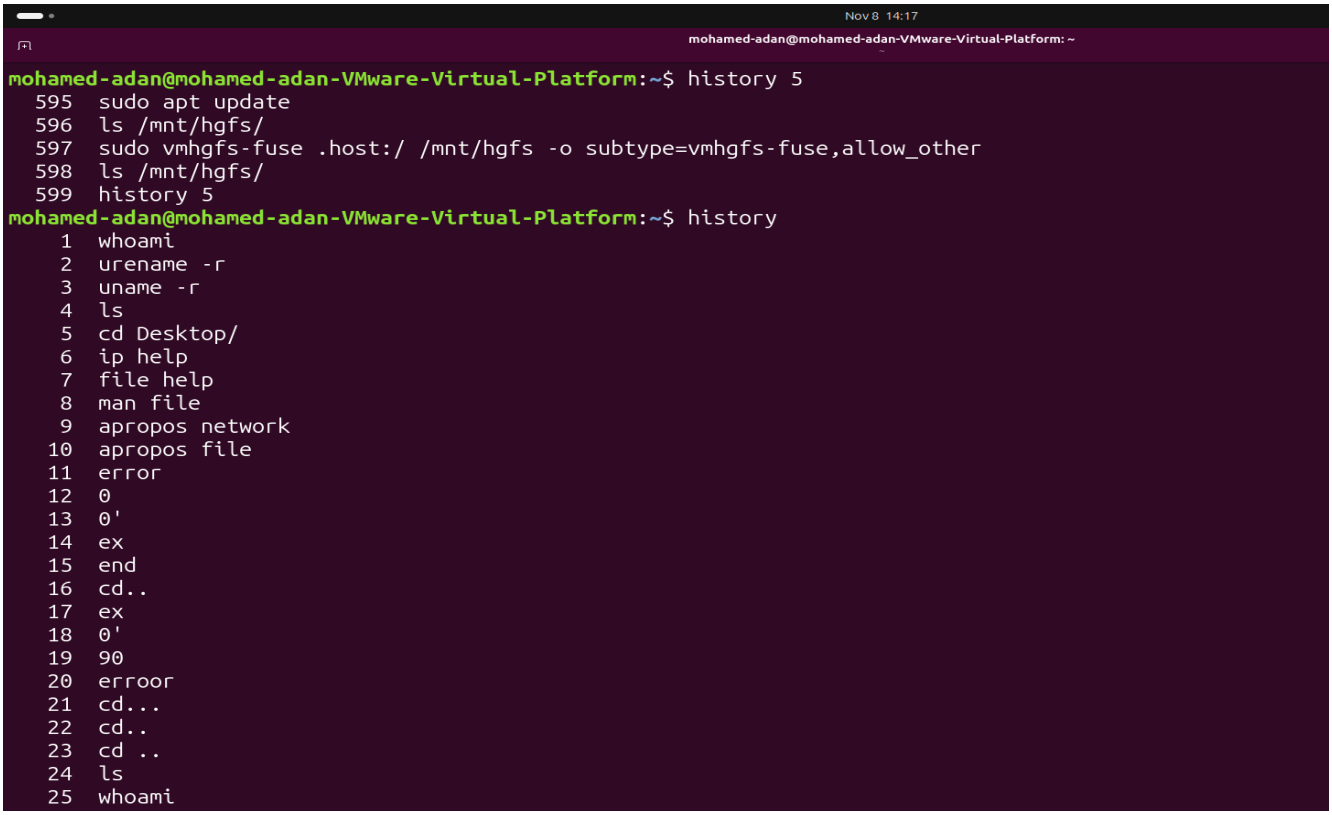


6. The history Command

The **history** command displays a numbered list of the commands you have recently typed and executed in your current terminal session. This is highly useful for reviewing past actions, finding a long command you don't want to retype, or keeping track of your workflow.

Options:

- **-c:** (Clear) Clears the entire command history for the current session.
- **-w:** (Write) Writes the current session's history to the history file (usually ~/.bash_history), saving it for future sessions.
- **-d offset:** (Delete) Deletes the history entry at the specific position number (offset).
- **[number]:** If you type history 5, it will only display the last 5 commands you ran (instead of the whole list).



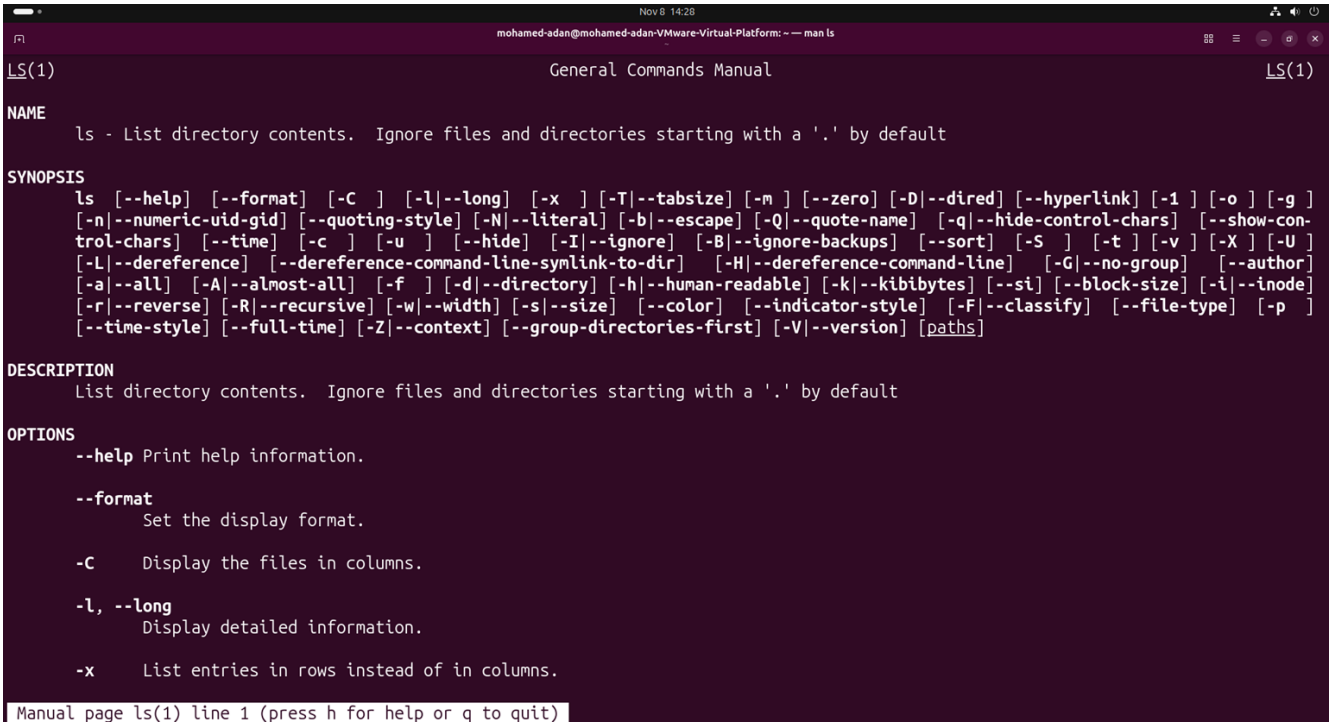
7. The man Command

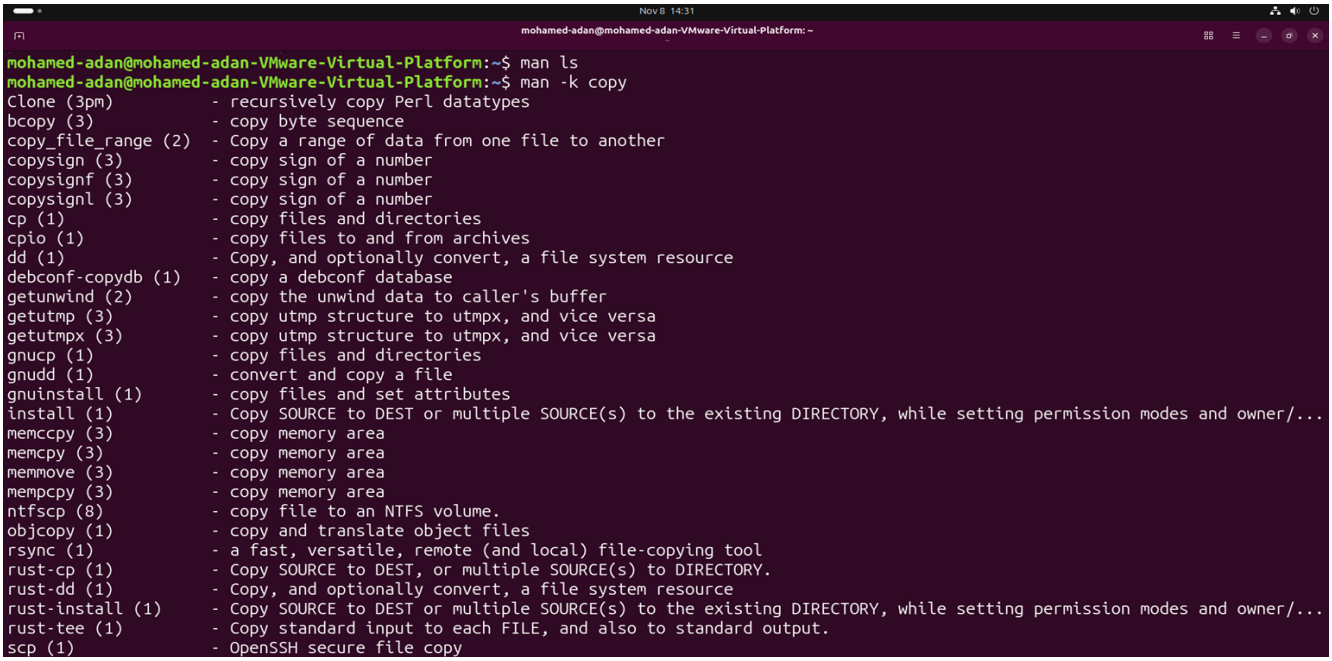
man stands for Manual. It is the built-in documentation system in Linux. When you forget how a command works or what options it has, man provides a detailed, formal manual page (called a "man page") right inside your terminal. It is the most reliable way to learn about Linux tools.

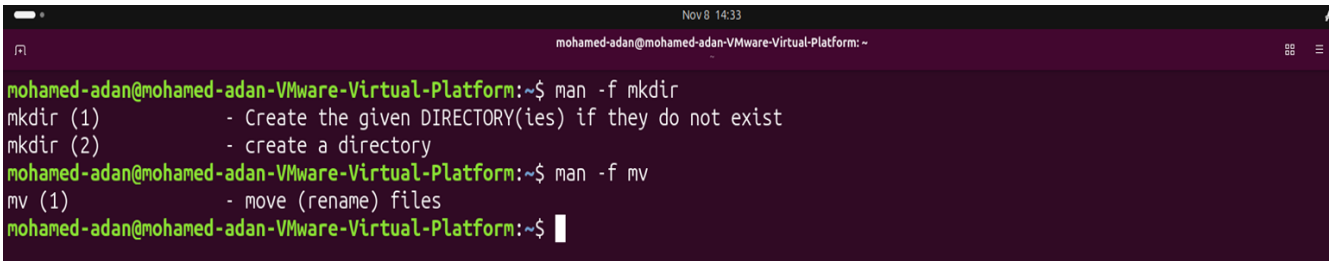
Options:

- **man [command]:** Opens the manual page for a specific command (e.g., man ls).
- **-k [keyword]:** Searches the short descriptions of all manual pages for a keyword. This is helpful if you don't know the exact command name (e.g., man -k copy).
- **-f [command]:** Displays a short, one-line description of what the command does.

Note: While reading a man page, use the Up/Down arrows to scroll, press q to quit, and press / to search inside the page.


Nov 8 14:28
mohamed-adan@mohamed-adan-VMware-Virtual-Platform: ~ — man ls
LS(1) General Commands Manual LS(1)
NAME
ls - List directory contents. Ignore files and directories starting with a '.' by default
SYNOPSIS
ls [--help] [--format] [-C] [-l|--long] [-x] [-T|--tabsize] [-m] [--zero] [-D|--dired] [--hyperlink] [-1] [-o] [-g]
[-n|--numeric-uid-gid] [--quoting-style] [-N|--literal] [-b|--escape] [-Q|--quote-name] [-q|--hide-control-chars] [--show-con-
trol-chars] [--time] [-c] [-u] [--hide] [-I|--ignore] [-B|--ignore-backups] [--sort] [-S] [-t] [-v] [-X] [-U]
[-L|--dereference] [--dereference-command-line-symlink-to-dir] [-H|--dereference-command-line] [-G|--no-group] [--author]
[-a|--all] [-A|--almost-all] [-f] [-d|--directory] [-h|--human-readable] [-k|--kibibytes] [--si] [--block-size] [-i|--inode]
[-r|--reverse] [-R|--recursive] [-w|--width] [-s|--size] [--color] [--indicator-style] [-F|--classify] [--file-type] [-p]
[--time-style] [--full-time] [-Z|--context] [--group-directories-first] [-V|--version] [paths]
DESCRIPTION
List directory contents. Ignore files and directories starting with a '.' by default
OPTIONS
--help Print help information.
--format
Set the display format.
-C Display the files in columns.
-l, --long
Display detailed information.
-x List entries in rows instead of in columns.
Manual page ls(1) line 1 (press h for help or q to quit)


Nov 8 14:31
mohamed-adan@mohamed-adan-VMware-Virtual-Platform: ~
mohamed-adan@mohamed-adan-VMware-Virtual-Platform:~$ man ls
mohamed-adan@mohamed-adan-VMware-Virtual-Platform:~$ man -k copy
Clone (3pm) - recursively copy Perl datatypes
bcopy (3) - copy byte sequence
copy_file_range (2) - Copy a range of data from one file to another
copysign (3) - copy sign of a number
copysignf (3) - copy sign of a number
copysignl (3) - copy sign of a number
cp (1) - copy files and directories
cpio (1) - copy files to and from archives
dd (1) - Copy, and optionally convert, a file system resource
debconf-copydb (1) - copy a debconf database
getunwind (2) - copy the unwind data to caller's buffer
getutmp (3) - copy utmp structure to utmpx, and vice versa
getutmpx (3) - copy utmp structure to utmpx, and vice versa
gnucp (1) - copy files and directories
gnudd (1) - convert and copy a file
gnuinstall (1) - copy files and set attributes
install (1) - Copy SOURCE to DEST or multiple SOURCE(s) to the existing DIRECTORY, while setting permission modes and owner/...
memccpy (3) - copy memory area
memcpy (3) - copy memory area
memmove (3) - copy memory area
mempcpy (3) - copy memory area
ntfscp (8) - copy file to an NTFS volume.
objcopy (1) - copy and translate object files
rsync (1) - a fast, versatile, remote (and local) file-copying tool
rust-cp (1) - Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.
rust-dd (1) - Copy, and optionally convert, a file system resource
rust-install (1) - Copy SOURCE to DEST or multiple SOURCE(s) to the existing DIRECTORY, while setting permission modes and owner/...
rust-tee (1) - Copy standard input to each FILE, and also to standard output.
scp (1) - OpenSSH secure file copy

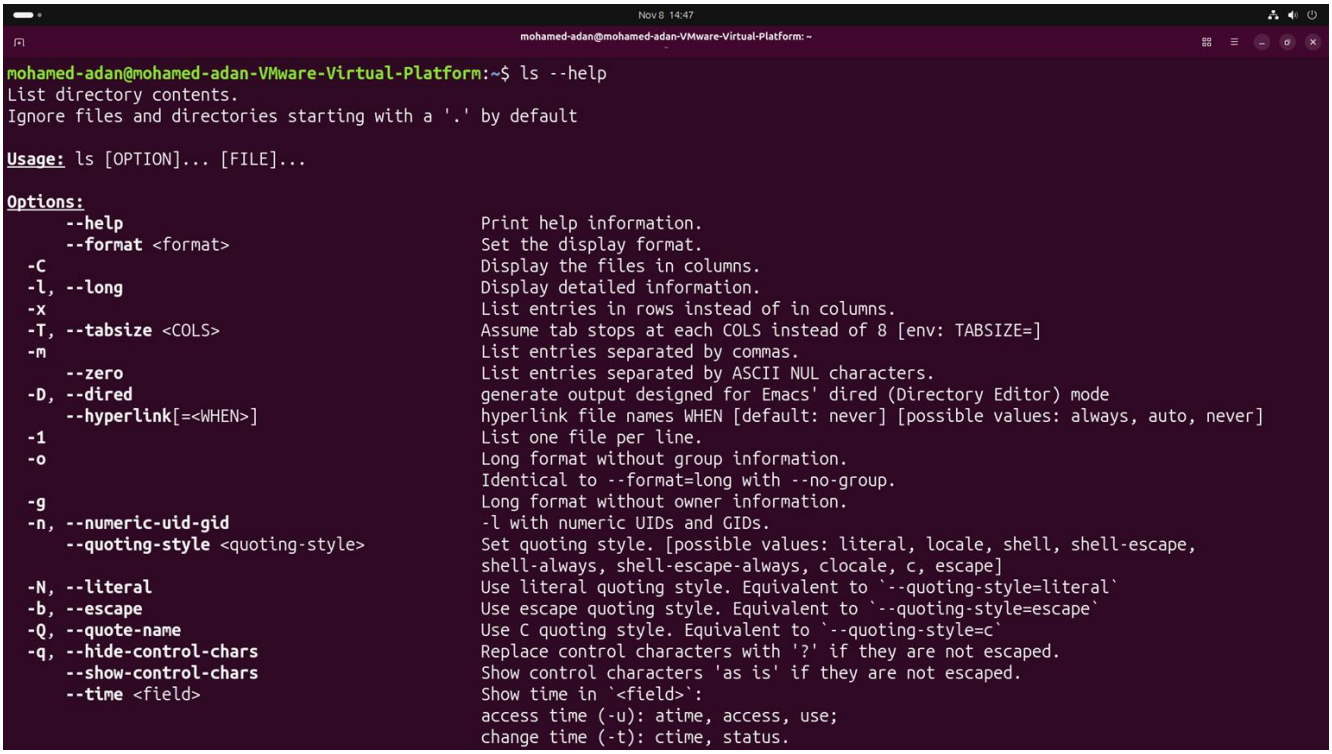

Nov 8 14:33
mohamed-adan@mohamed-adan-VMware-Virtual-Platform: ~
mohamed-adan@mohamed-adan-VMware-Virtual-Platform:~$ man -f mkdir
mkdir (1) - Create the given DIRECTORY(ies) if they do not exist
mkdir (2) - create a directory
mohamed-adan@mohamed-adan-VMware-Virtual-Platform:~$ man -f mv
mv (1) - move (rename) files
mohamed-adan@mohamed-adan-VMware-Virtual-Platform:~$

8. The --help Option

While man is the full manual, almost every Linux command has a built-in **--help** option. This is a shortcut that prints a quick summary of how to use the command and its most common options directly to the screen. It is faster than reading a full man page when you just need a quick reminder.

Options:

- **--help:** Appends to the end of a command to show its quick guide (e.g., ls --help).
- **-h:** On many commands, a single -h also works as a shortcut for help.

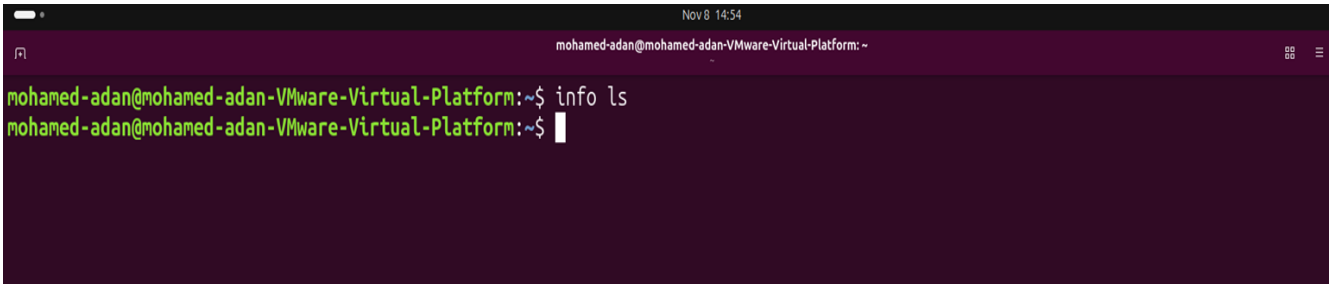


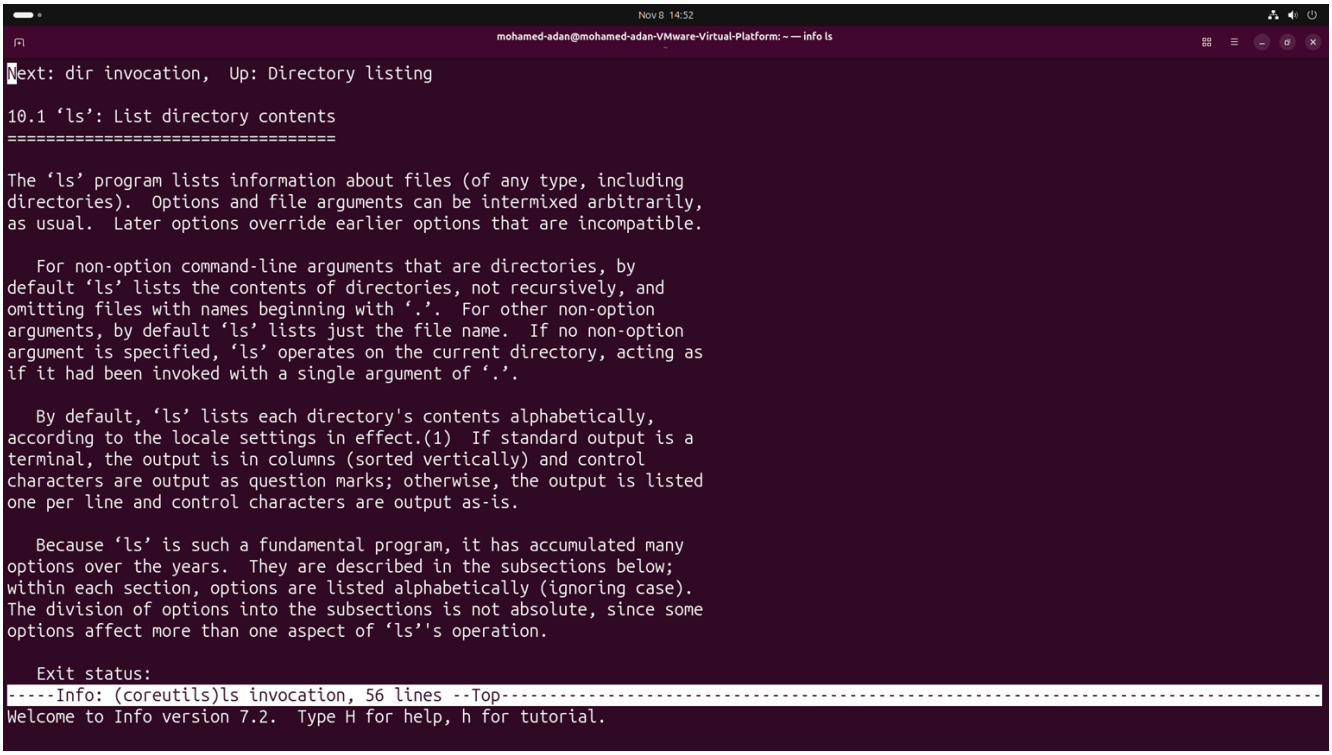
9. The info Command

The **info** command is an alternative to man. It provides documentation in a hyperlinked, menu-driven format. While man pages are just walls of text, info pages are broken down into "nodes" (like chapters in a book) that you can navigate using your keyboard. It is often used for complex tools like GNU software.

Options:

- **info [command]:** Opens the info page for a command (e.g., info ls).
- **Navigation:** Use Space to scroll down, Backspace to scroll up, and q to quit.

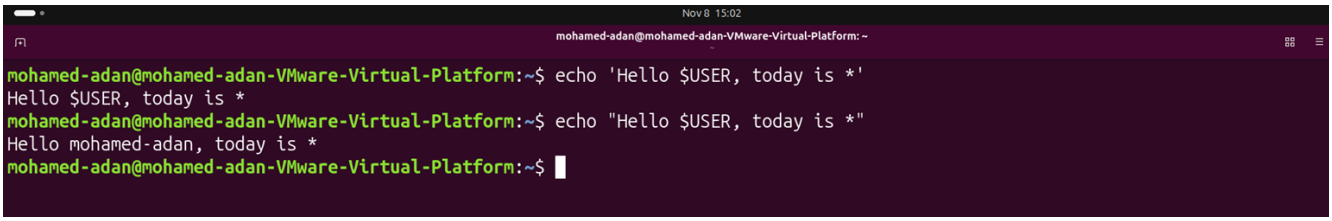




10. Single Quotes (' ') & Double Quotes (" ")

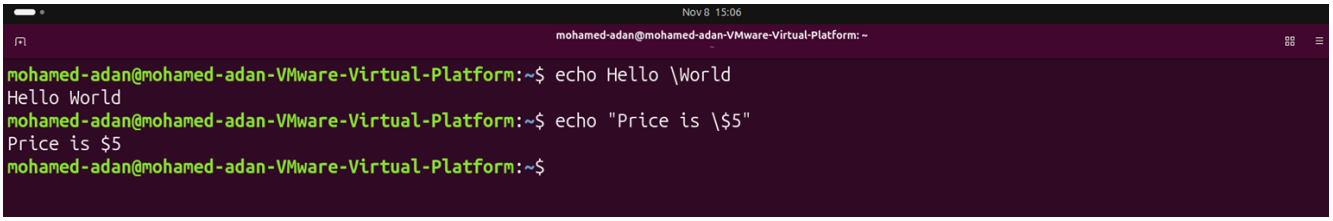
Single quotes are used to tell the terminal to treat everything inside them as literal text. The shell ignores any special characters inside single quotes. If you put a $ or a * inside single quotes, it will just be printed as a normal character rather than being executed.

Double quotes are similar to single quotes, but they are "weaker." They treat most characters as literal text, but they allow variable expansion. If you put a $USER inside double quotes, the terminal will replace it with your actual username. However, they still protect against wildcards (*) and spaces.



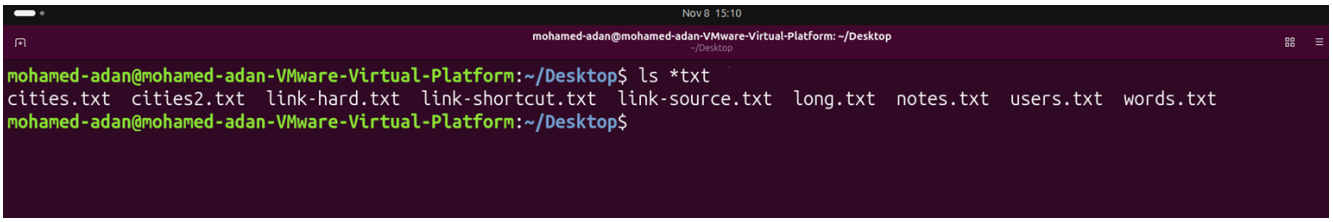
11. Escaping (\)

Escaping means **using a backslash (\)** to "escape" the special meaning of a character. When you put a \ directly before a special character (like a space, $, or *), it tells the terminal to treat that character as normal text. It is most commonly used to include spaces in file or folder names.



12. Wildcards (*)

The **asterisk (*)** is a wildcard (or "glob") that represents zero or more characters. The terminal uses it to match patterns, which is extremely useful for finding or listing multiple files that share a similar name.



13. Variables (Environment Variables)

In Linux, **a variable** is a named storage location that holds data. Environment variables are special variables used by the shell and programs to know how the system is configured. For example, the **$USER** variable holds your username, and **$HOME** holds the path to your home directory. You can create your own variables and use them in commands.

Note: Notice how putting the **$ symbol** before the name calls the value inside the variable.

Note: There are NO spaces around the = sign. This is very important in Linux!



14. PATH (Finding Programs)

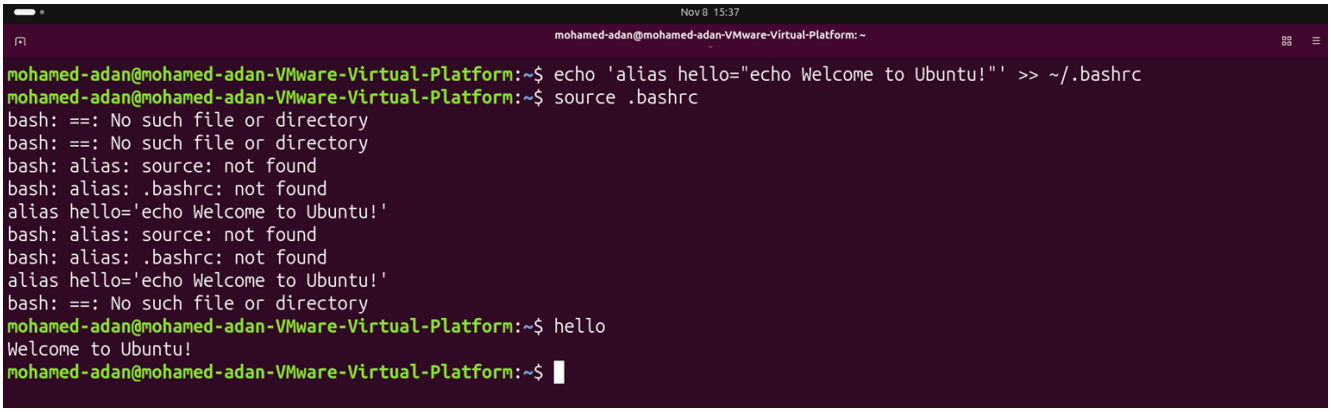
The **$PATH** variable is one of the most important environment variables. It contains a list of directories (folders) where the system should look for executable programs. When you type a command like ls or pwd, Linux doesn't magically know where those programs are; it searches through the folders listed in your $PATH until it finds the program.

Note: which is a great command that tells you exactly which folder in the PATH contains the program you are asking for



15. The ~/.bashrc File

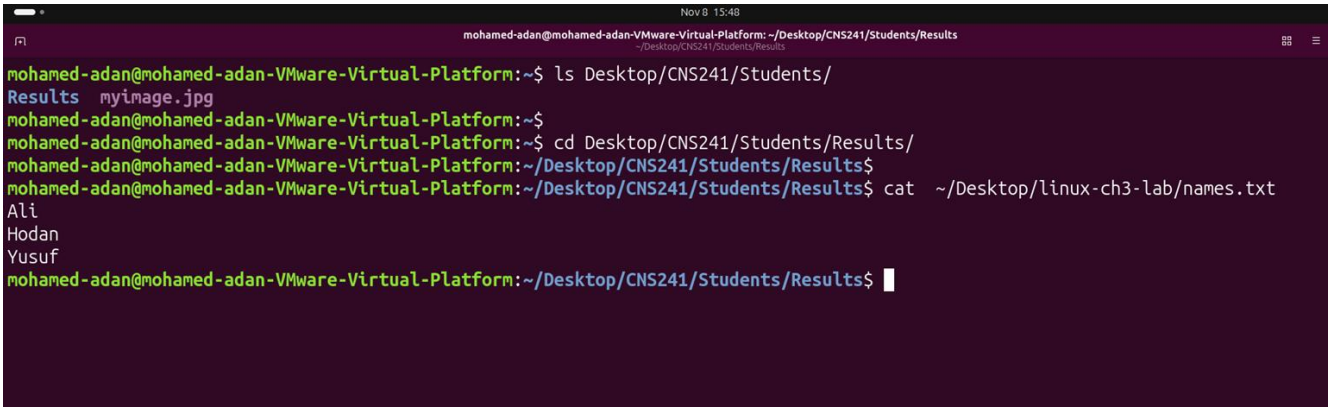
The file **~/.bashrc** (the ~ means your home directory, and the . means it is a hidden file) is a configuration script that runs every time you open a new terminal. It is used to set up your environment, create shortcut commands (aliases), and customize your terminal.



16. Absolute Paths & Relative Paths

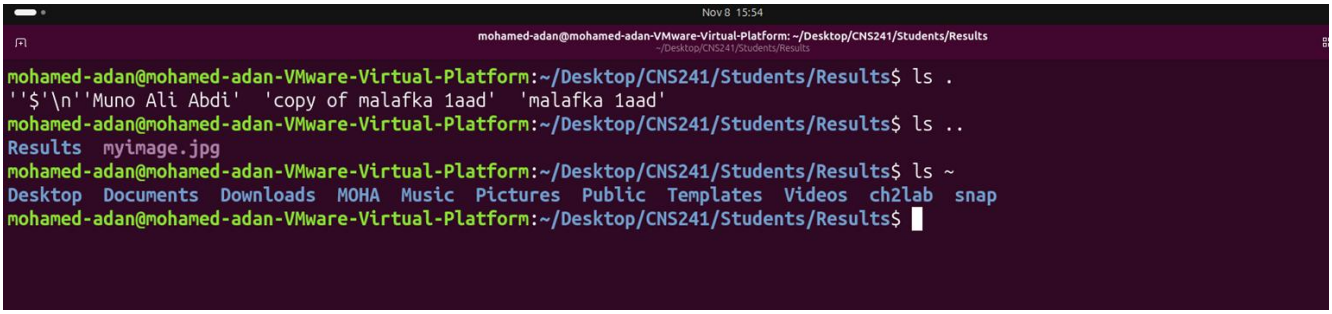
An **absolute path** is a path that starts from the very top of the file system (the root directory, represented by a forward slash /). It is like a full mailing address: it tells the system exactly how to get to a file or directory from the root, regardless of where you are currently located. Absolute paths always start with a /.

A **relative path** is a path that is interpreted based on your current working directory. It does **not start with a /**. Instead, it tells the system how to get to a file relative to where you are right now. If you are in your home folder, typing cd Documents (without a slash) uses a relative path.



17. The Dot (.) , The Double Dot (..) & The Tilde (~)

1. A **single dot .** represents your current directory. It is often used in commands when you want to refer to something in the folder you are currently standing in.
2. The **double dot ..** represents the parent directory (the folder one level above your current location). This is the most common way to navigate backward in the file system.
3. The **tilde ~** is a shortcut that represents your home directory (e.g., /home/mohamed-adan). No matter where you are in the file system, typing cd ~ will instantly teleport you back to your personal home folder. It saves you from typing out the full absolute path.

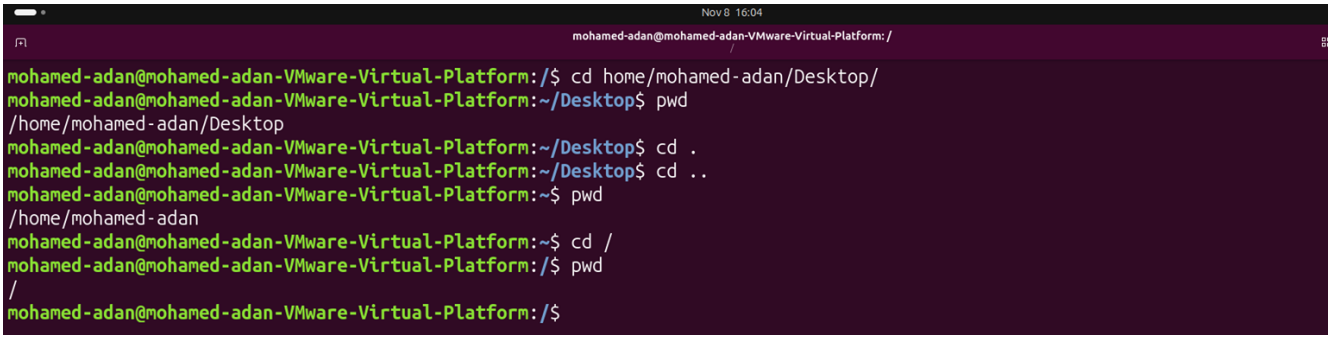


18. The cd Command

cd stands for Change Directory. It is the primary command used to navigate the Linux file system. It allows you to move from your current folder to a different one.

Options:

- **cd [directory]:** Moves into the specified directory.
- **cd ~:** Moves to your home directory.
- **cd ..:** Moves up one level to the parent directory.

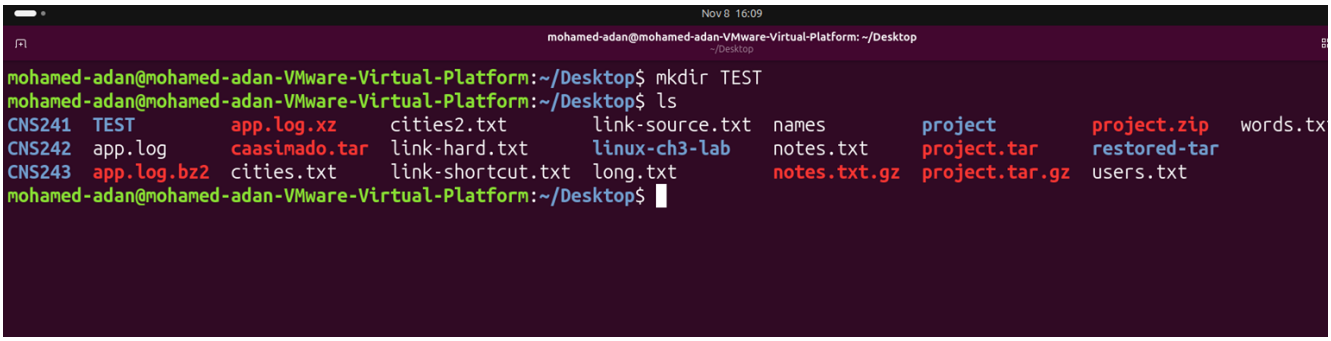


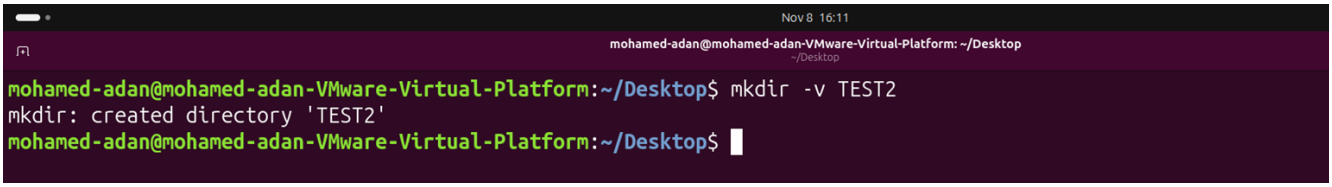
19. The mkdir Command

mkdir stands for Make Directory. It is used to create new, empty folders in the file system.

Options:

- **mkdir [name]:** Creates a single directory.
- **-v:** (Verbose) Prints a message for each directory created.





20. The touch Command

The **touch** command is primarily used to create new, empty files. If the file already exists, touch updates its "last modified" timestamp to the current time without changing the file's contents.

Options:

- **touch [filename]:** Creates an empty file.

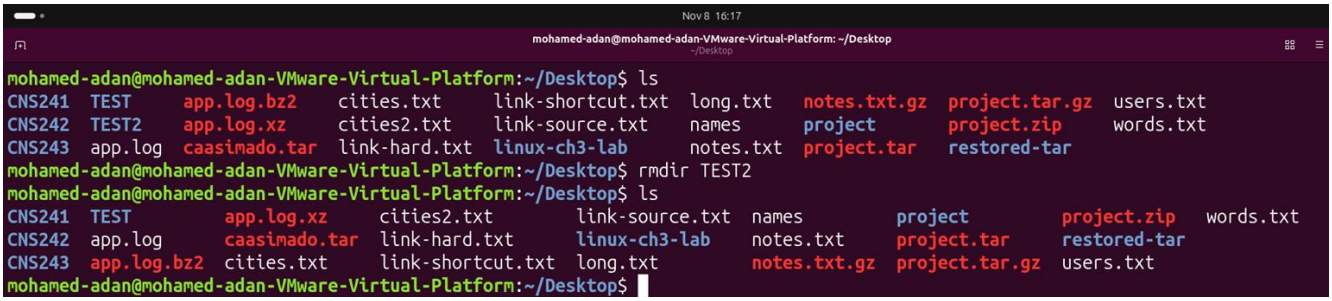


21. The rmdir Command

rmdir stands for Remove Directory. It is used to delete empty directories. It is a safe command because it will refuse to delete a folder if there are any files or subfolders inside it.

Options:

- **rmdir [directory]:** Deletes an empty directory.

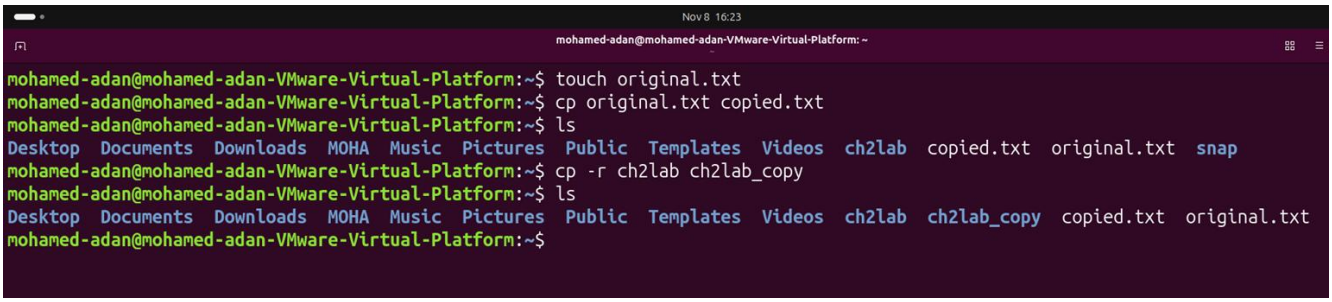


22. The cp Command

cp stands for Copy. It is used to duplicate files or directories from one location to another. The original file remains untouched.

Options:

- **cp [source] [destination]:** Copies a file.
- **-r:** (Recursive) Required to copy directories (folders) and everything inside them.
- **-v:** (Verbose) Shows exactly what is being copied.

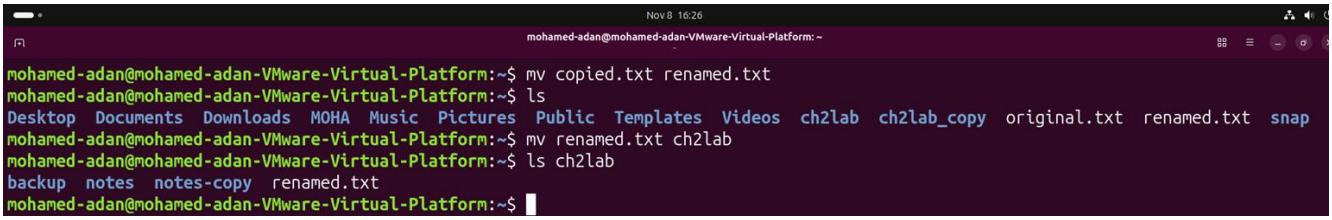


23. The mv Command

mv stands for Move. It is used for two purposes: moving files/folders to a new location, OR renaming them.

Options:

- **mv [source] [destination]:** Moves or renames a file.
- **-v:** (Verbose) Shows what is being moved.



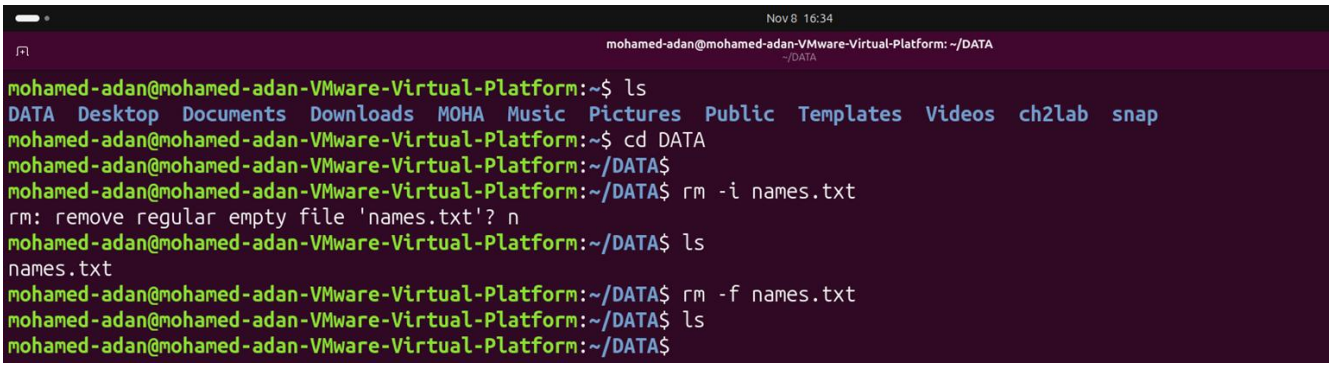
24. The rm Command

rm stands for Remove. It is used to permanently delete files and directories. Warning: In Linux, rm does not use a "Trash" or "Recycle Bin"—once you delete a file with rm, it is gone forever.

Options:

- **rm [file]:** Deletes a file.
- **-r:** (Recursive) Deletes a directory and all its contents.
- **-i:** (Interactive) Prompts you for confirmation before deleting each file (safest option).
- **-f:** (Force) Ignores non-existent files and never prompts for confirmation.





25. The mkdir -p Command (Parent Directories)

By default, mkdir can only create a folder if its parent folder already exists. The **-p** (parents) option allows you to create a full nested directory structure in one single command. If the parent folders don't exist, it will create them automatically.

Options:

- **mkdir -p folder/subfolder/subsubfolder:** Creates the whole chain.

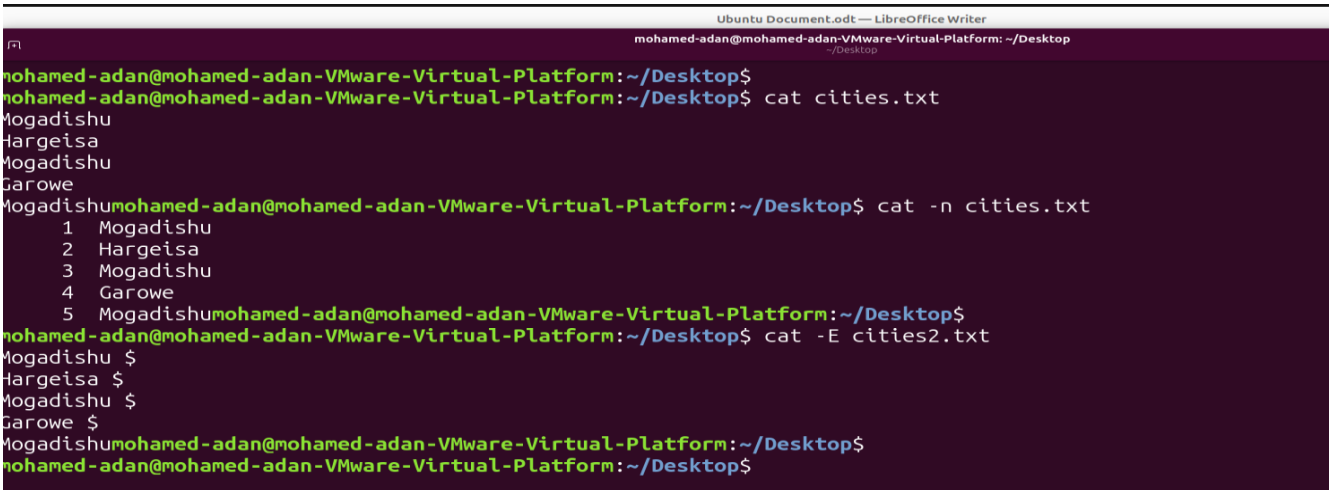


26. The cat Command

cat stands for Concatenate. It is one of the most basic commands used to read files and print their entire contents directly to the terminal. It is best used for short files.

Options:

- **cat [filename]:** Prints the file to the screen.
- **-n:** Numbers all output lines.
- **-E:** Displays a $ at the end of each line to show line breaks.

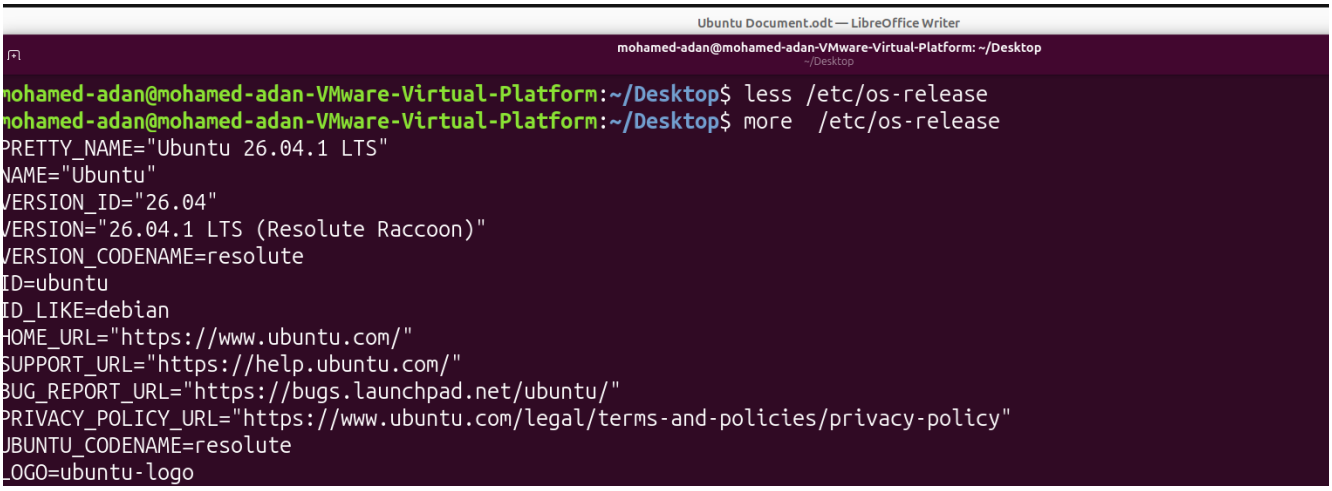


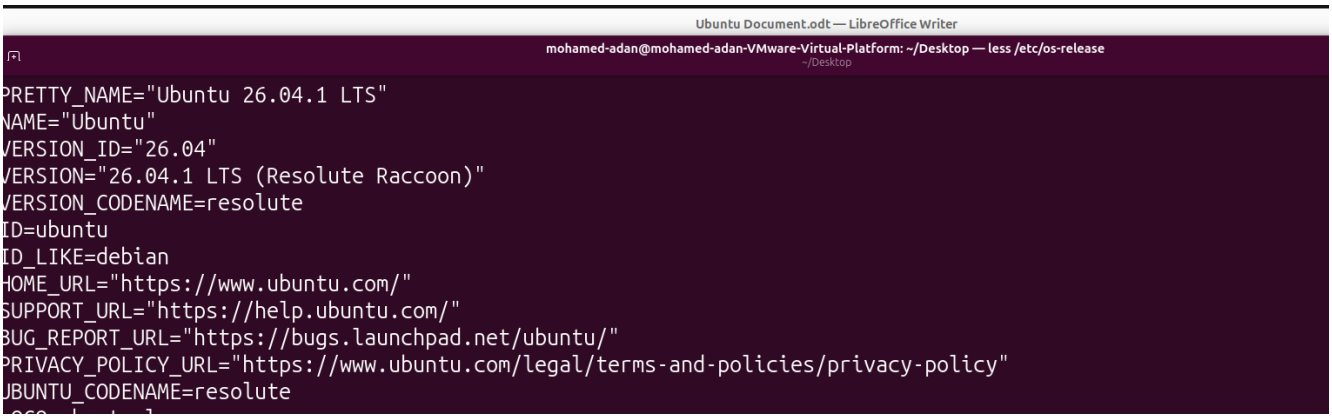
27. The less and more Commands

If a file is very long (like a system log), cat will flood your screen. **less and more** are pagers. They allow you to view large files one screen at a time. less is more advanced than more because it allows you to scroll backward and forward.

Options:

- **less [filename]:** Opens the file.
- **While inside less:** Use Space to go down, b to go up, and q to quit.
- **more [filename]:** Opens the file (but you can only scroll forward).



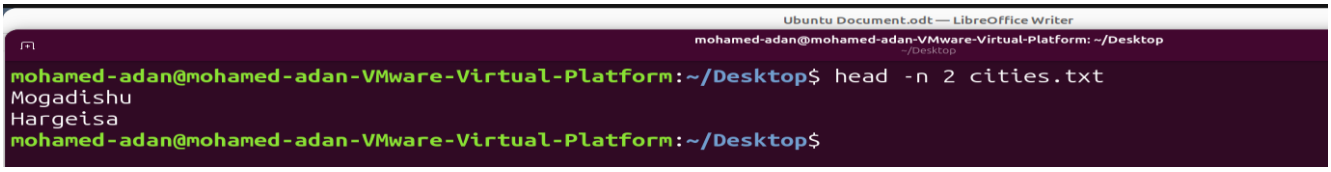


28. The head Command

head is used to view the top portion of a file. By default, it prints the first 10 lines.

Options:

- **-n [number]:** Specifies how many lines to print (e.g., -n 5 prints the first 5 lines)

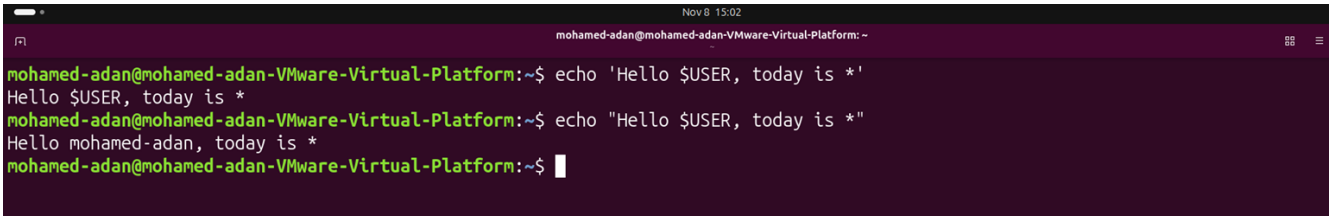


29. The tail Command

tail is the opposite of head. It is used to view the bottom portion of a file. By default, it prints the last 10 lines.

Options:

- **-n [number]:** Specifies how many lines to print from the bottom.
- **-f:** (Follow) This is a very important option for system administrators. It keeps the file open and prints new lines in real-time as they are added to the file.

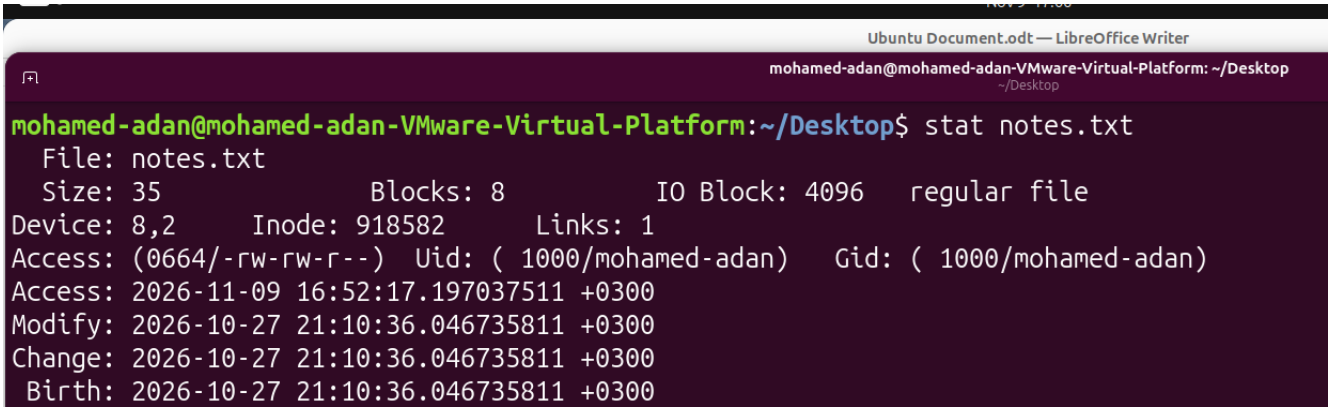


30. The stat Command

stat stands for Status. While ls -l gives you basic file info, stat gives you highly detailed metadata about a file, including exact timestamps (when it was accessed, modified, and changed), file size in bytes, and inode number.

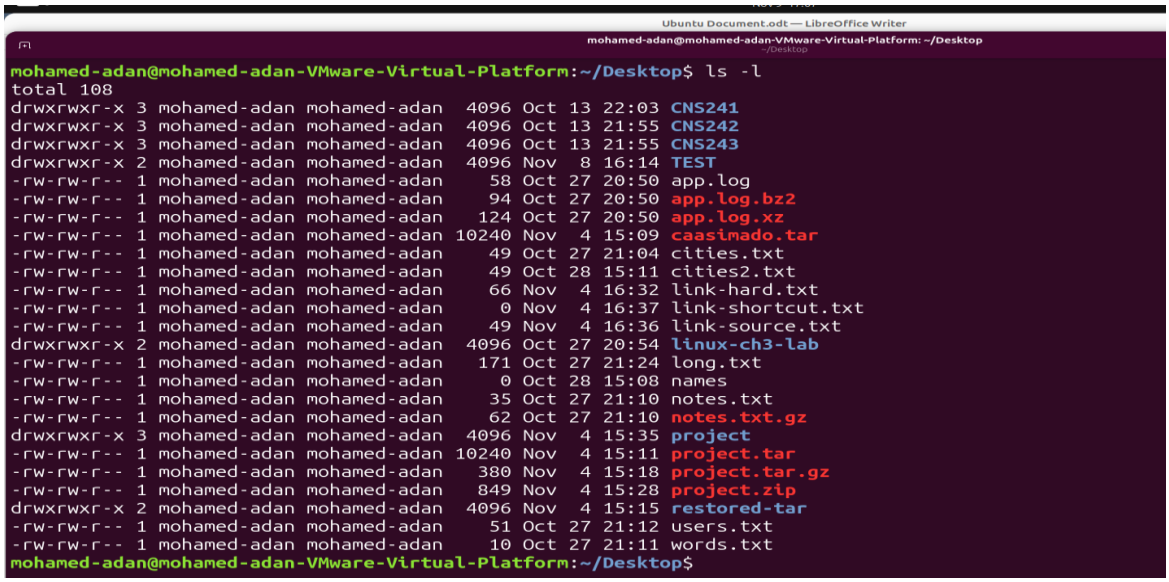
Options:

- **stat [filename]:** Displays the metadata.



31. The ls -l Command

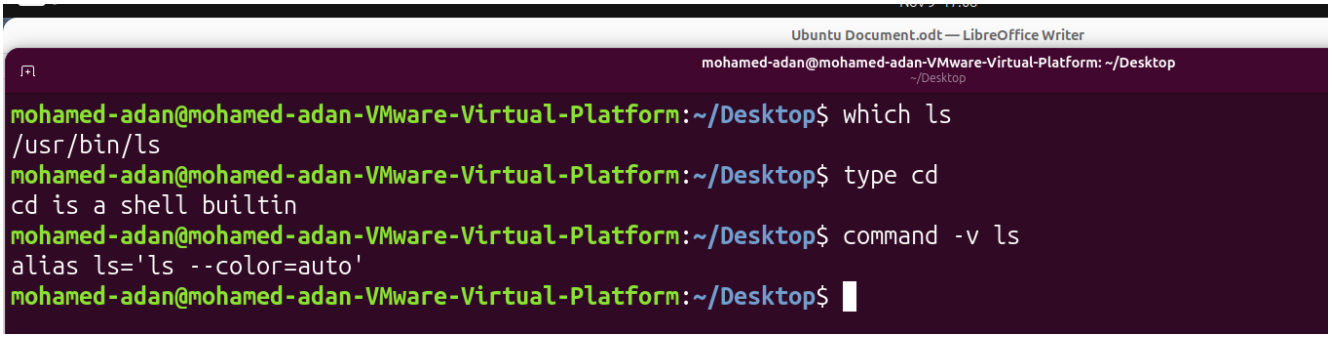
ls lists directory contents, but the -l (long) option formats the output to show detailed information: file permissions, number of links, owner, group, size, and modification date.



32. Locating Programs: which, type, and command -v

These three commands are used to find out **where a specific program is located** on your system.

1. **which:** Searches the $PATH variable for the executable file.
2. **type:** Tells you how the shell will interpret the command (e.g., is it a built-in shell command, an alias, or a file?).
3. **command -v:** A highly portable way to find the path of a command.

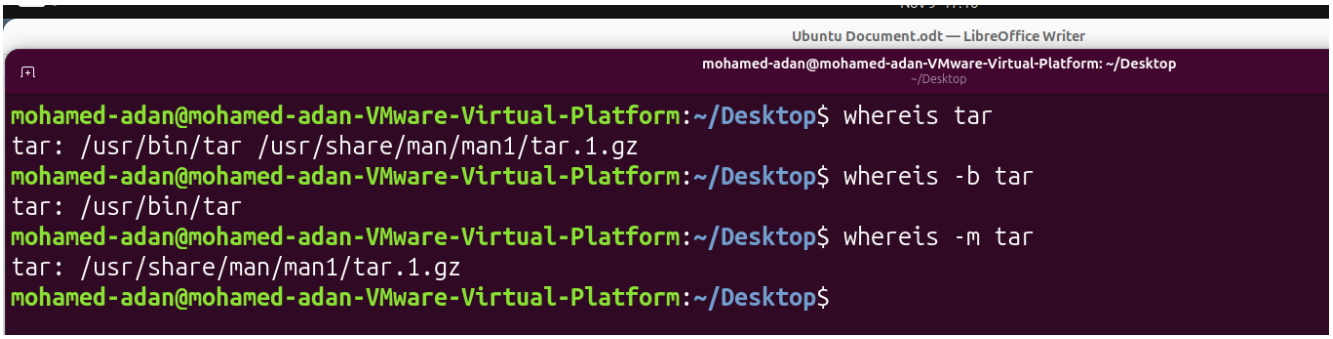


33. The whereis Command

whereis is broader than which. It locates the binary (executable) file, the source code, and the manual pages for a specific command.

Options:

- **-b:** Only search for binaries.
- **-m:** Only search for manuals.

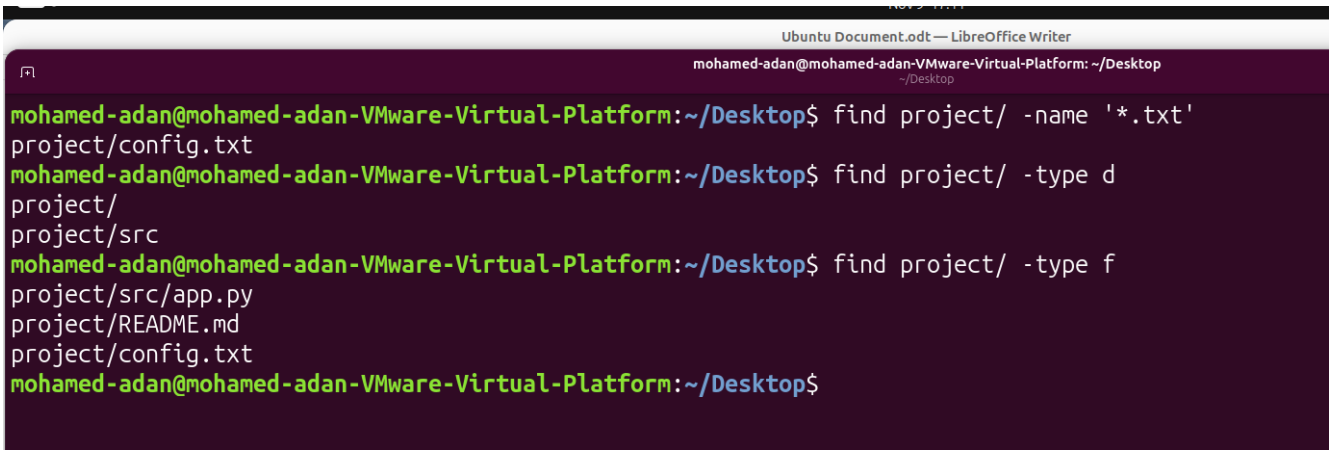


34. The find Command (Searching by Name)

find is an extremely powerful command used to search for files and directories based on various criteria. You can search by name, size, type, and more. It searches recursively through the file system.

Options:

- **-name "pattern":** Searches for files matching the exact name or pattern (wildcards like * are allowed).
- **-type f:** Searches only for files.
- **-type d:** Searches only for directories.



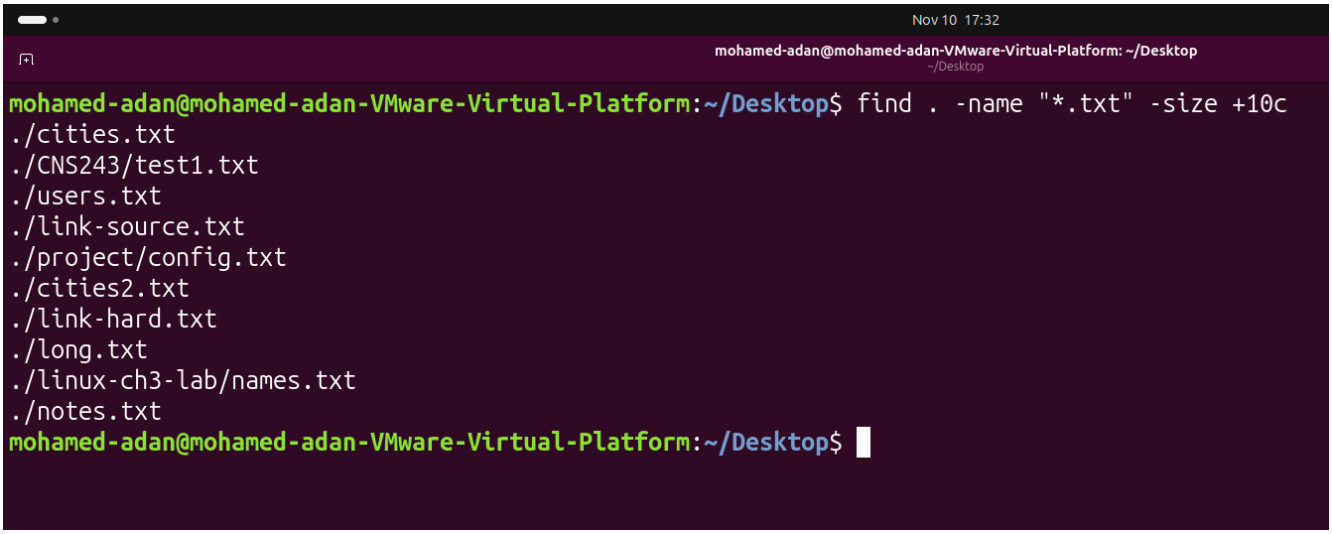
35. The find Command (Searching by Size)

You can also use find to locate files **based on their size**. This is useful for finding large files that are taking up disk space.

Note: c stands for bytes, k for kilobytes, M for megabytes. A + means "greater than", a - means "less than".

Options:

- **-size +10c:** Finds files larger than 10 bytes.



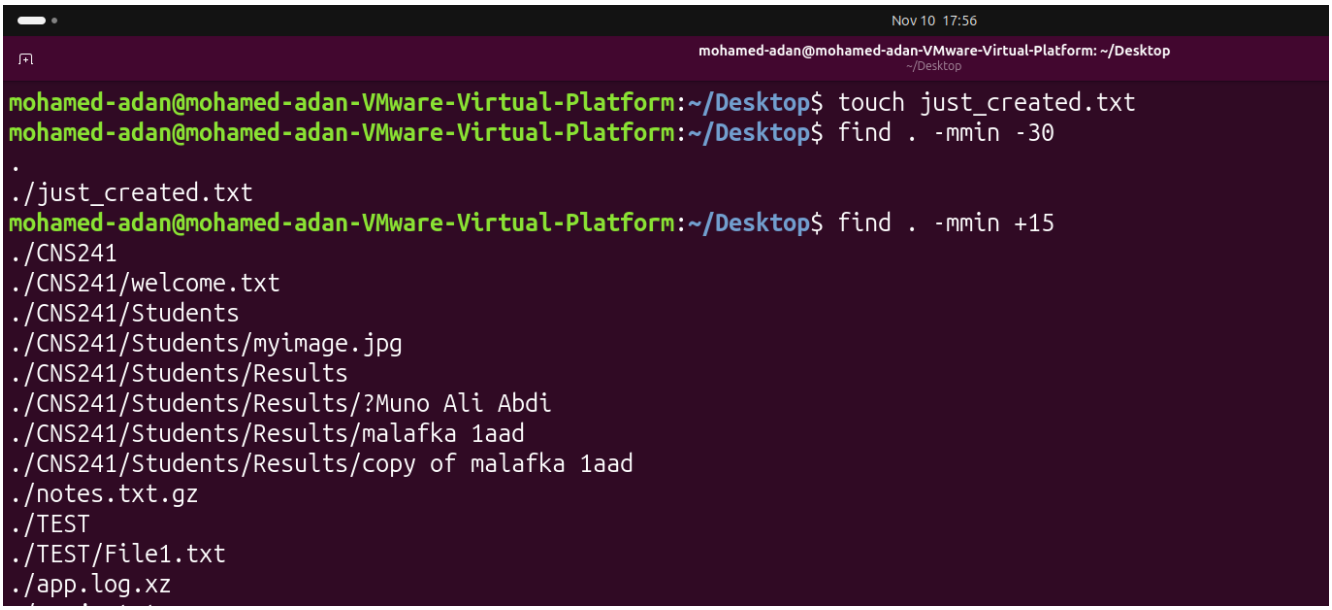
36. The find Command (Searching by Time)

In addition to names and sizes, find can search for files based on when they were last **modified.** The **-mmin** option stands for "modification time in minutes." This is highly useful for finding files you were just working on.

Options:

- **-mmin -30:** Finds files modified less than 30 minutes ago.
- **-mmin +15:** Finds files modified more than 30 minutes ago.

- **-mtime -1:** Finds files modified less than 1 day (24 hours) ago.



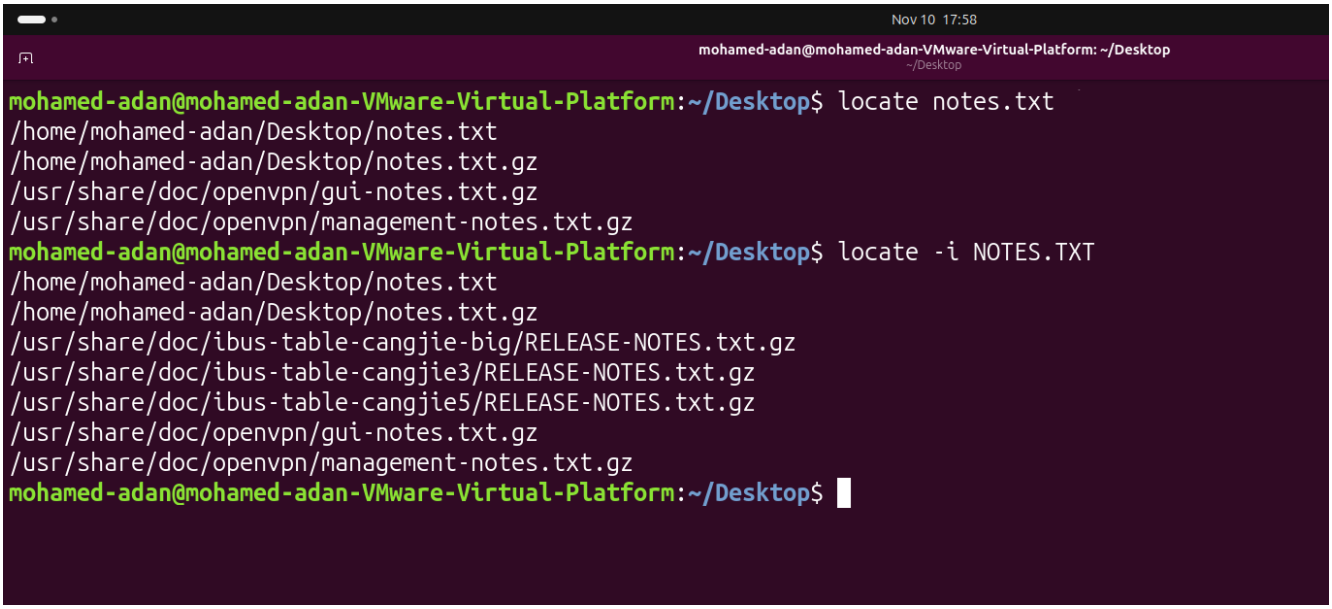
37. The locate Command

While find searches your hard drive in real-time (which can be slow), **locate** searches a pre-built database of files. It is incredibly fast. However, it only knows about files that existed the last time the database was updated (usually done automatically once a day).

Options:

- **locate [filename]:** Finds the file instantly.
- **-i:** Ignores case (uppercase/lowercase).

Note: If locate doesn't work, you may need to run **sudo updatedb** to build the database.

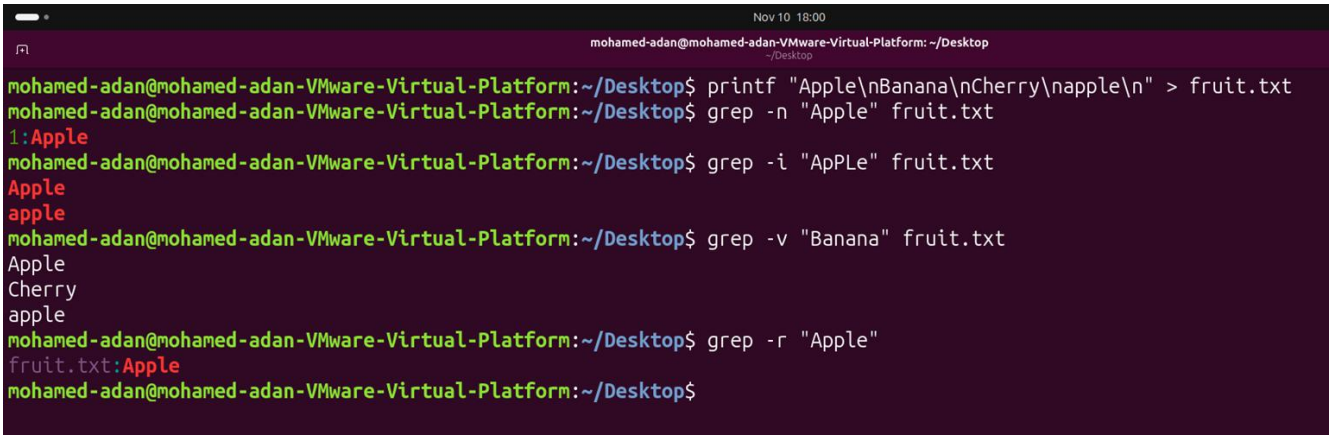


38. The grep Command (Basic Search)

grep stands for Global Regular Expression Print. It is one of the most famous Linux commands. It searches through text inside files or output from other commands and prints any line that matches a specific pattern.

Options:

- **-n:** Shows the line number where the match is found.
- **-i:** Ignores case (finds "Linux", "linux", "LINUX").
- **-r:** (Recursive) Searches through all files in a directory and its subdirectories.
- **-v:** (Invert) Shows lines that do not contain the match.
- **-c (Count):** Instead of showing the matches, it shows the number of matching lines.



39. Regular Expressions with grep

Regular Expressions (Regex) are special characters that create advanced search patterns. **grep** uses these to find complex text patterns rather than just exact words.

The 5 Core Regex Concepts:

- **The Dot (.):** Matches any single character. (e.g., gr.y matches "gray" and "grey").
- **The Asterisk (*):** Matches zero or more of the preceding character. (e.g., Aple matches "Ale", "Apple", "Appple").
- **The Caret (^):** Matches the beginning of a line.
- **The Dollar Sign ($):** Matches the end of a line.
- **Brackets ([ ]):** Matches any one character inside the brackets. [0-9] matches any number, and gr[ae]y matches both "gray" and "grey".



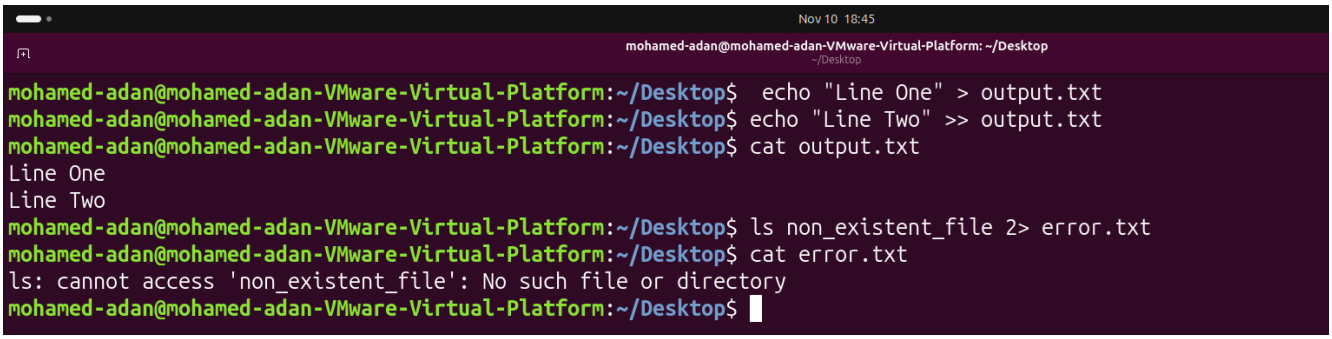
40. Standard Streams and Redirection (>, >>, 2>, <)

Linux has three "**Standard Streams**" for input and output:

1. **Standard Input (stdin - 0):** What you type or feed into a program.
2. **Standard Output (stdout - 1):** The normal result of a command.
3. **Standard Error (stderr - 2):** Error messages.

Redirection allows you to take the output (or error) and send it somewhere else instead of printing it on the screen.

- **> :** Redirects Standard Output to a file. (Warning: This overwrites the file!).
- **>> :** Redirects Standard Output and appends (adds) to the end of the file.
- **2> :** Redirects Standard Error to a file.
- **< :** Redirects Standard Input. It feeds a file into a command as input.



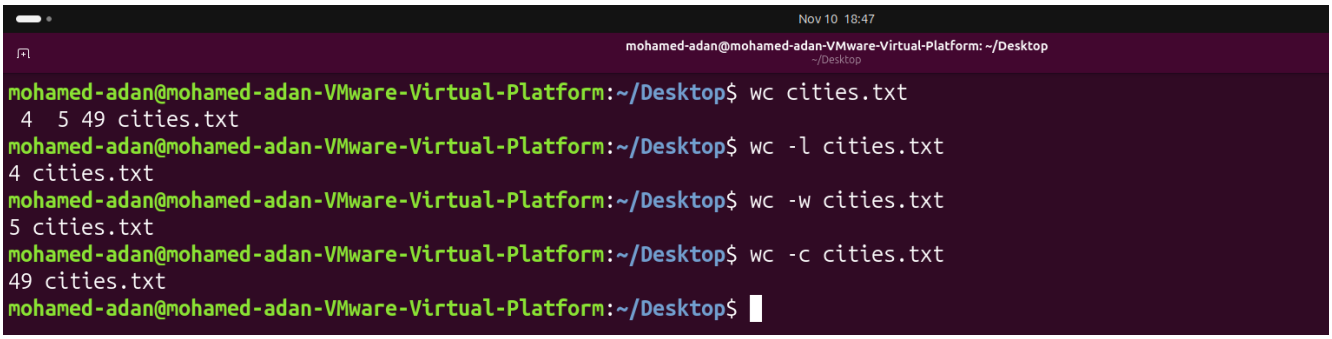
41. Counting Lines, Words, and Bytes (wc)

wc stands for Word Count. It reads a file (or output from a pipe) and counts the number of lines, words, and bytes.

Options:

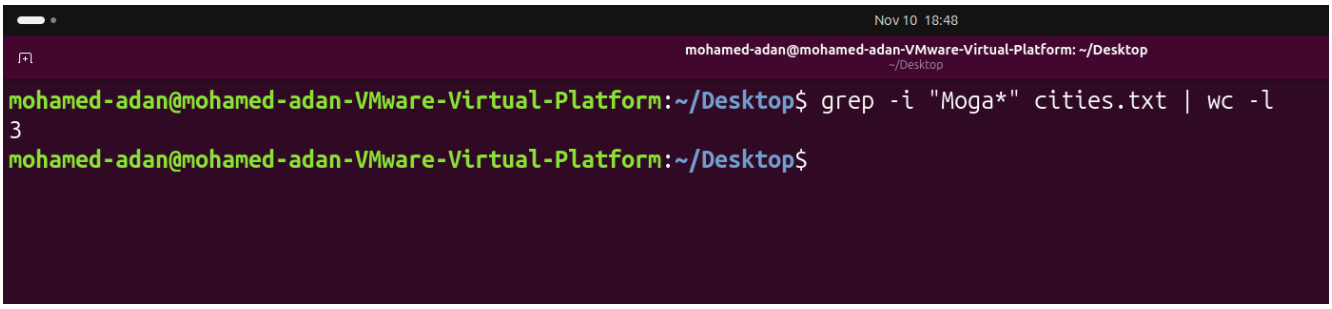
- **-l:** Counts only the lines.
- **-w:** Counts only the words.
- **-c:** Counts only the bytes (characters).

- **-m:** Counts the characters (useful for multi-byte characters).



42. The Pipe (|)

The pipe **symbol |** is one of the most powerful tools in Linux. It takes the Standard Output of the command on the left and feeds it directly as the Standard Input to the command on the right. It allows you to chain commands together.



Note: This tells the system: "Search for 'Line', and instead of printing the results, **pass them to wc -l** to count them."

43. Conditional Chaining (&&, || and ;)

Instead of pipes, you can use **logical operators** to run commands based on whether the previous command succeeded or failed.

- **&& (AND):** Runs the second command only if the first command succeeds.
- **|| (OR):** Runs the second command only if the first command fails.
- **; (Semicolon - Unconditional):** Runs the first command, and no matter what happens (success or failure), it will immediately run the second command. It acts like hitting "Enter" between commands.

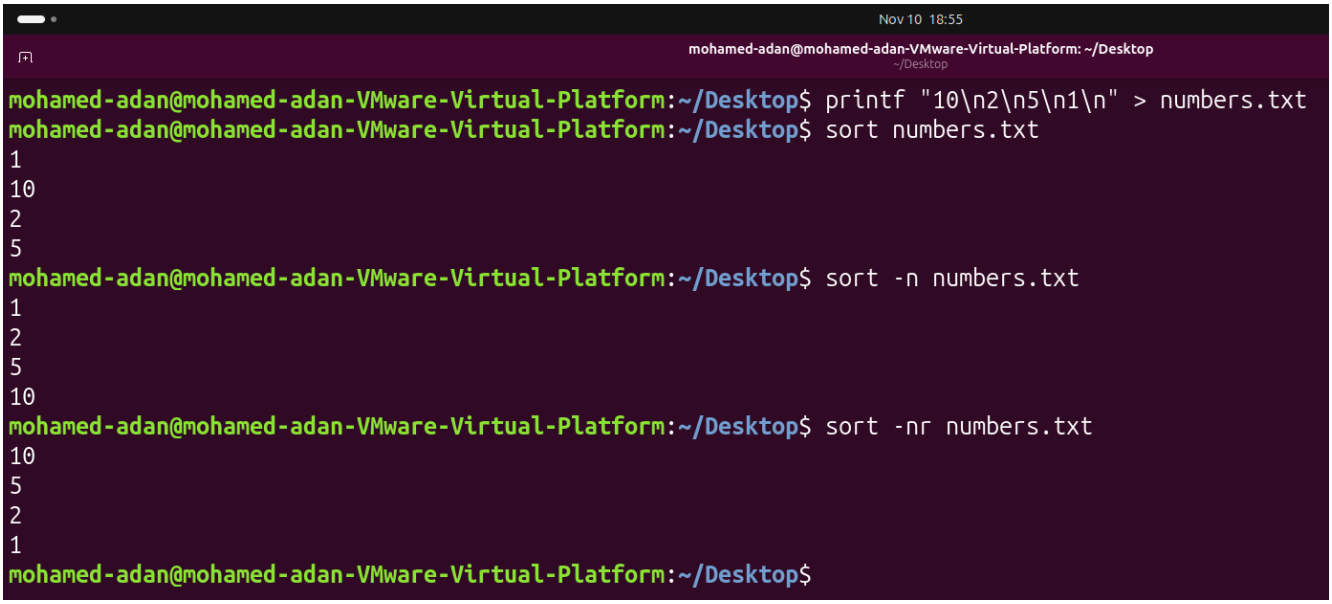


44. The sort Command

sort arranges the lines of a text file in alphabetical or numerical order.

Options:

- **-n:** Sorts numerically (so "10" comes after "2", not before).
- **-r:** Reverses the order (descending instead of ascending).



45. The uniq Command

uniq stands for Unique. It filters out adjacent duplicate lines. (Note: Because it only removes adjacent duplicates, it is almost always used after sort).

Options:

- **-c:** (Count) Prefixes lines with the number of times they occurred.
- **-d:** (Duplicates) Only shows lines that are repeated.

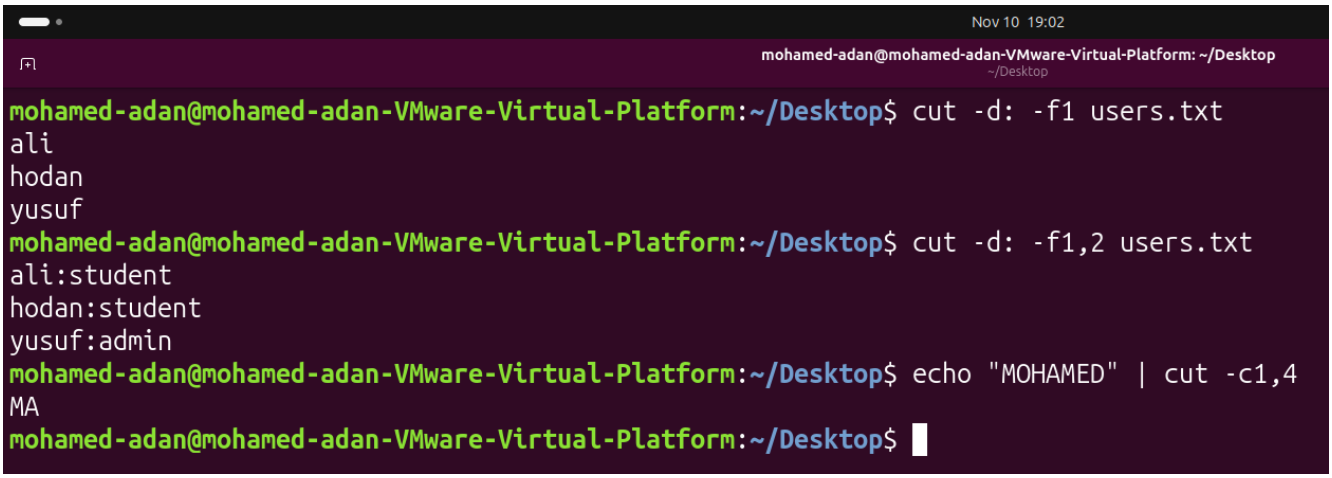


46. Extracting Text (cut)

cut removes sections from each line of a file. It is used to extract specific columns or characters.

Options:

- **-d:** Specifies the delimiter (the character that separates the columns, like a comma).
- **-f:** Specifies which field (column) to extract.
- **-c:** Cuts by specific character positions.

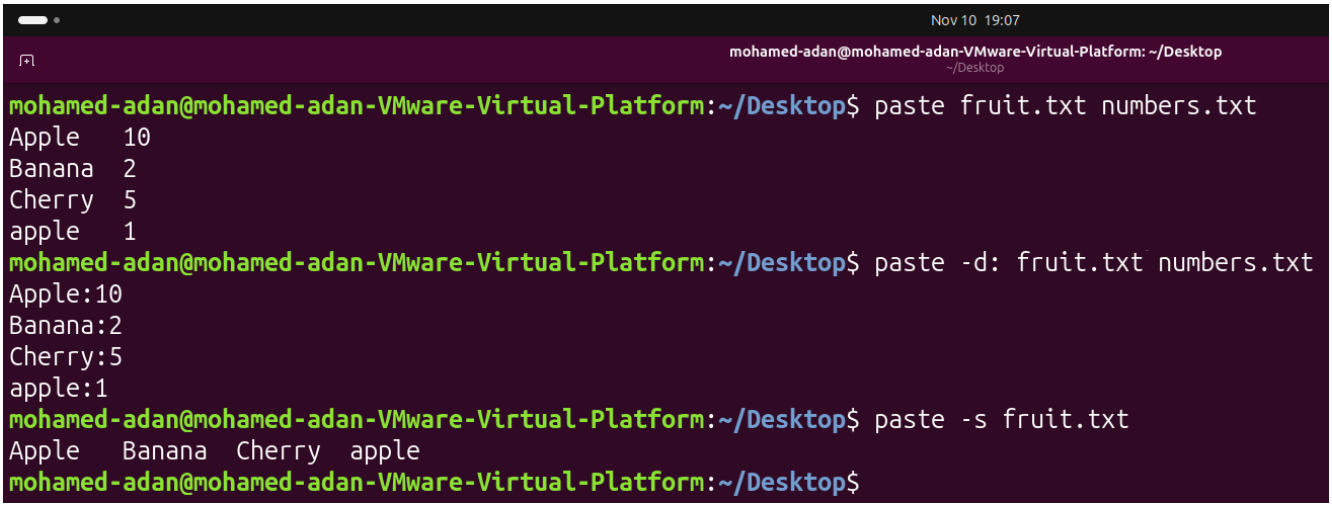


47. Joining Files (paste)

paste merges lines from multiple files together horizontally. It is the opposite of cat, which merges files vertically.

Options:

- **-d:** Specifies the delimiter to use between the pasted columns (default is a Tab).
- **-s:** (Serial) Pastes all lines from one file into a single line.

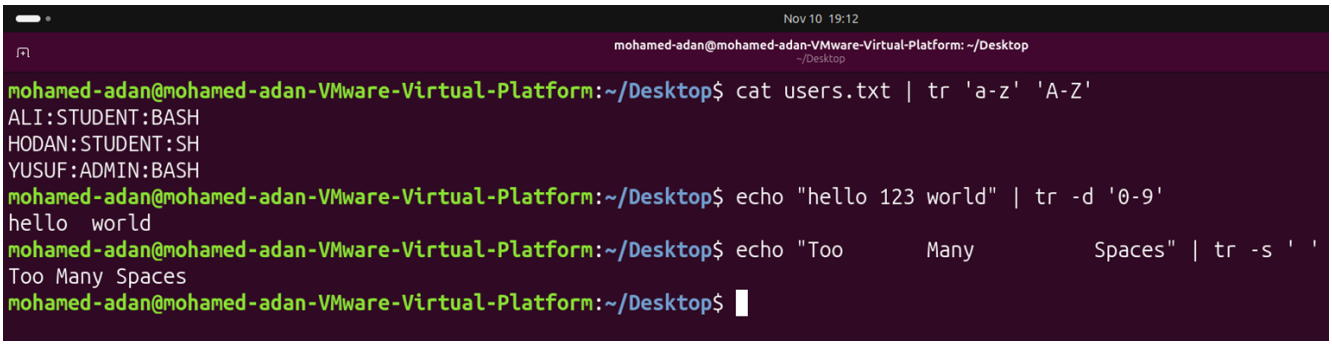


48. The tr Command (Translate)

tr stands for Translate. It reads text and replaces or deletes specific characters. It does not read from files directly; it must receive input from a pipe or < redirection.

Options:

- **-d:** (Delete) Deletes the specified characters.
- **-s:** (Squeeze) Replaces multiple consecutive occurrences of a character with a single occurrence.

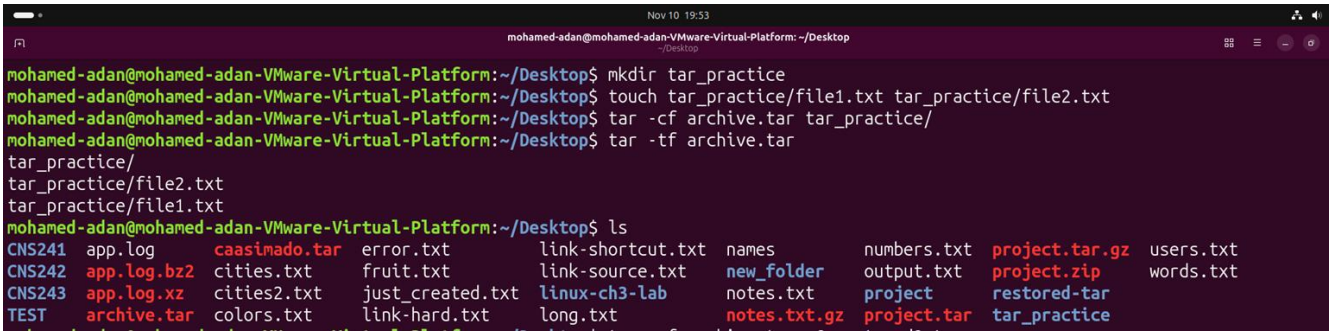


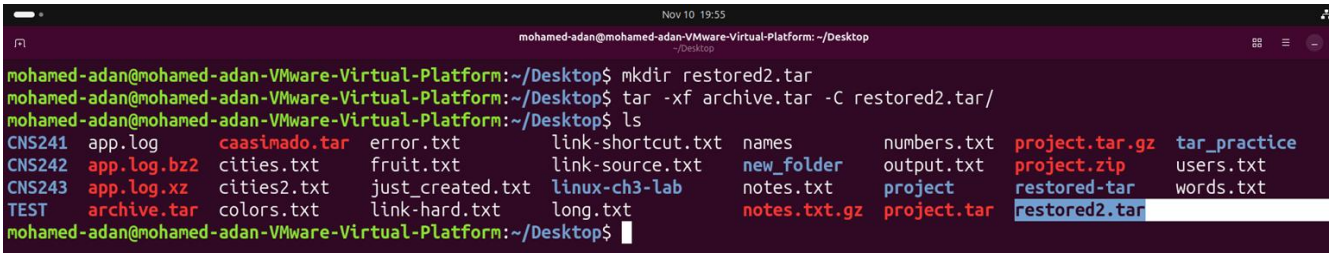
49. The tar Command (Create and Extract)

tar stands for Tape Archive. It is the standard tool in Linux for bundling multiple files and directories into a single file (often called a "tarball").

Options:

- **-c:** (Create) Creates a new archive.
- **-f:** (File) Specifies the name of the archive file. (This must always be followed by the archive name).
- **-v:** (Verbose) Shows the files being processed on the screen.
- **-x:** (Extract) Extracts the files from an archive.



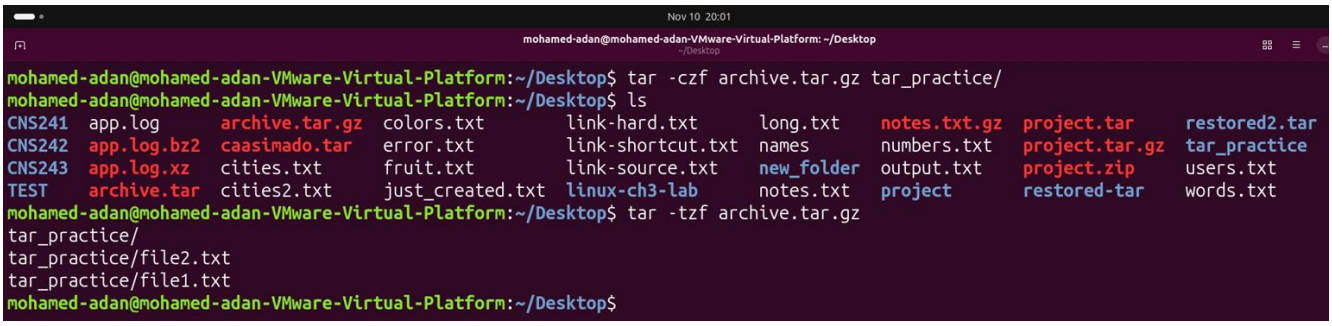


50. The tar Command (Compression Combinations)

tar can also compress the archive as it creates it, saving disk space. It uses different algorithms (gzip, bzip2, xz) which are activated by adding specific letters to the -c flag.

Options:

- **-czf:** Creates a gzip compressed archive (fast, standard .tar.gz).
- **-cjf:** Creates a bzip2 compressed archive (slower, smaller .tar.bz2).
- **-cJf:** Creates an xz compressed archive (slowest, smallest .tar.xz).g
- **-tzf:** Lists the contents of a gzip archive without extracting it.



51. gzip, gunzip, and xzcat

These commands **compress and decompress** individual files (not archives).

1. **gzip:** Compresses a file (replaces the original file with a .gz version).
2. **gunzip:** Decompresses a .gz file.
3. **xzcat:** Views the contents of an .xz compressed file without extracting it to your hard drive.

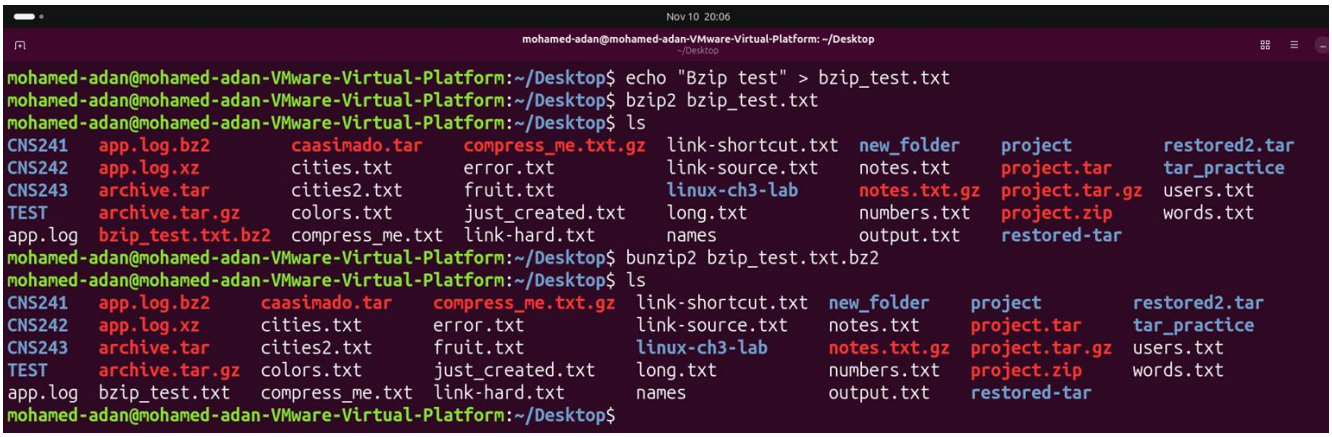
Options:

- **gzip -k:** (Keep) Compresses the file but keeps the original.
- **gzip -9:** Sets the slowest but maximum compression level (1-9).
- **gunzip -c:** Writes the decompressed output to the terminal instead of creating a new file.



52. bzip2 and bunzip2

Similar to **gzip**, these commands compress and decompress individual files, but they use the **bzip2** algorithm, which generally produces smaller files than gzip at the cost of speed.

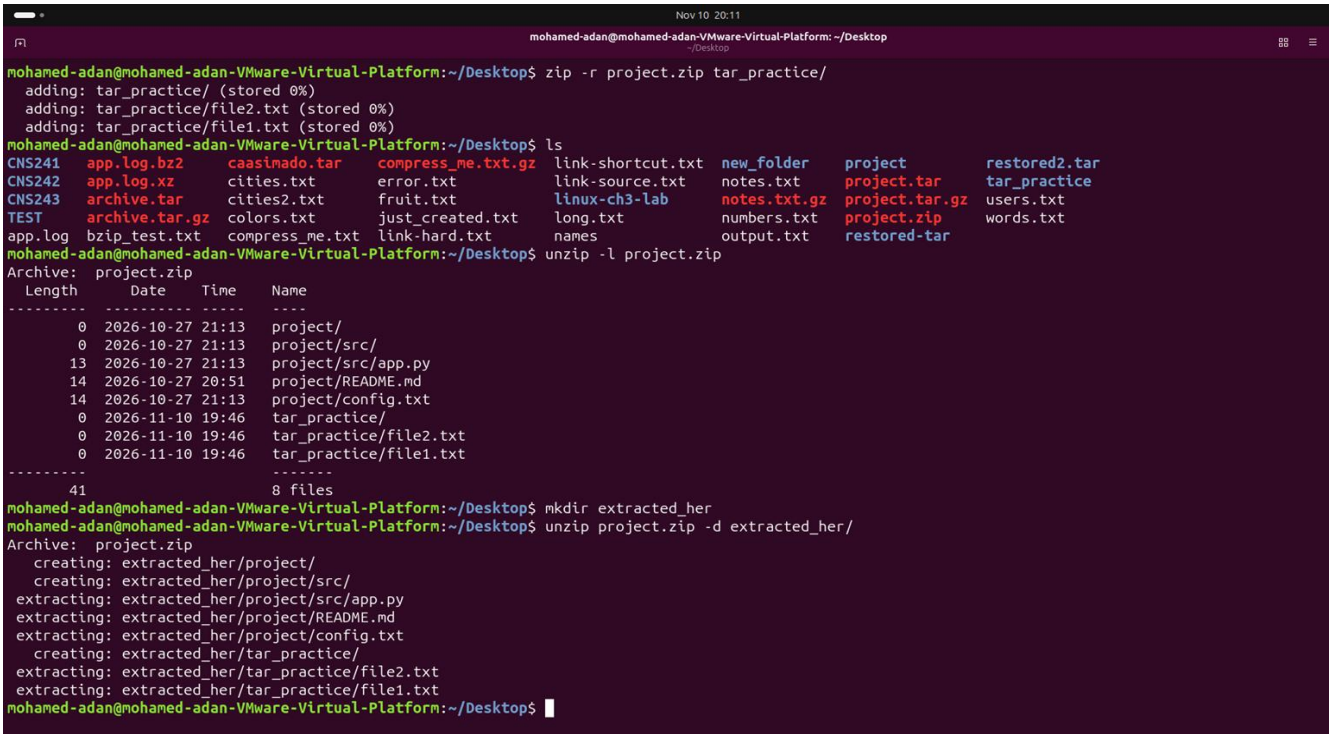


53. The zip and unzip Commands

While **tar** is the Linux standard, **zip** is used to create compressed archives that are highly compatible with Windows and macOS.

Options:

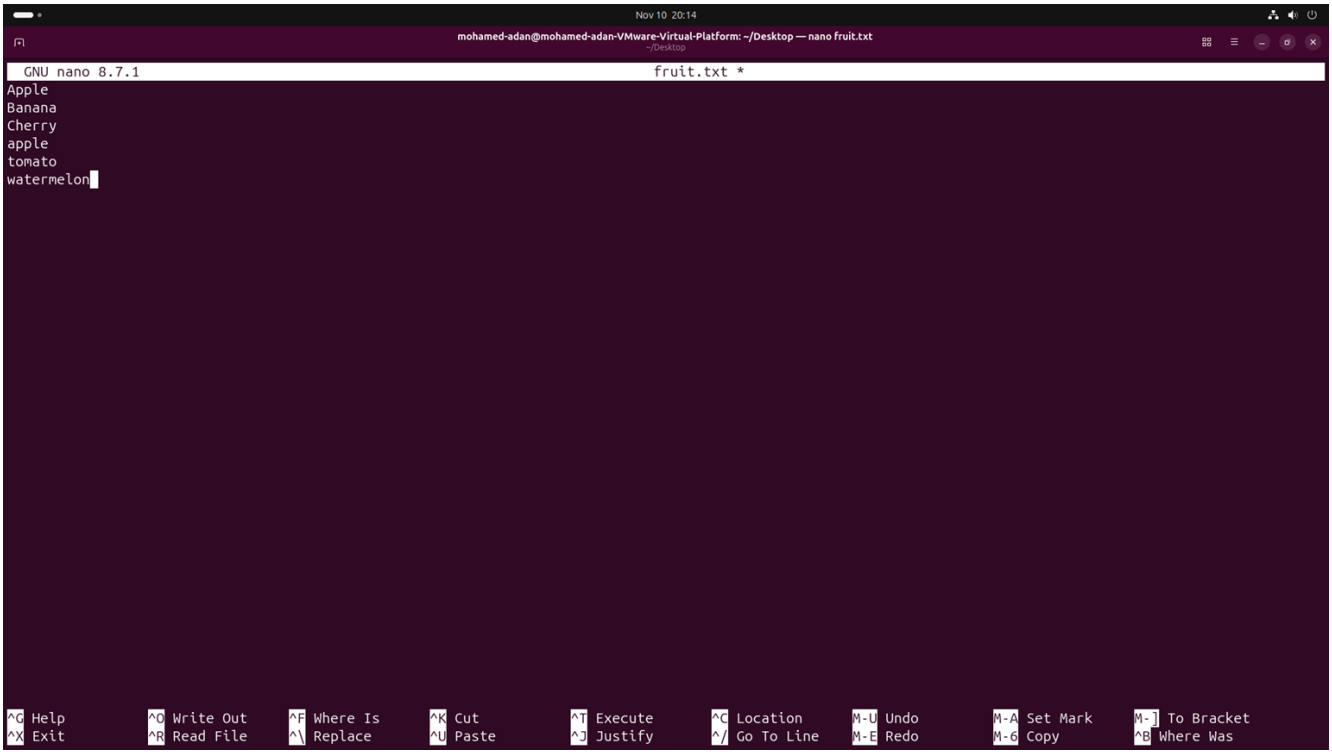
- **zip -r:** (Recursive) Zips a directory and all its contents.
- **unzip -l:** (List) Shows the contents of a zip file without extracting.
- **unzip -d:** (Directory) Extracts the zip file into a specific target directory.



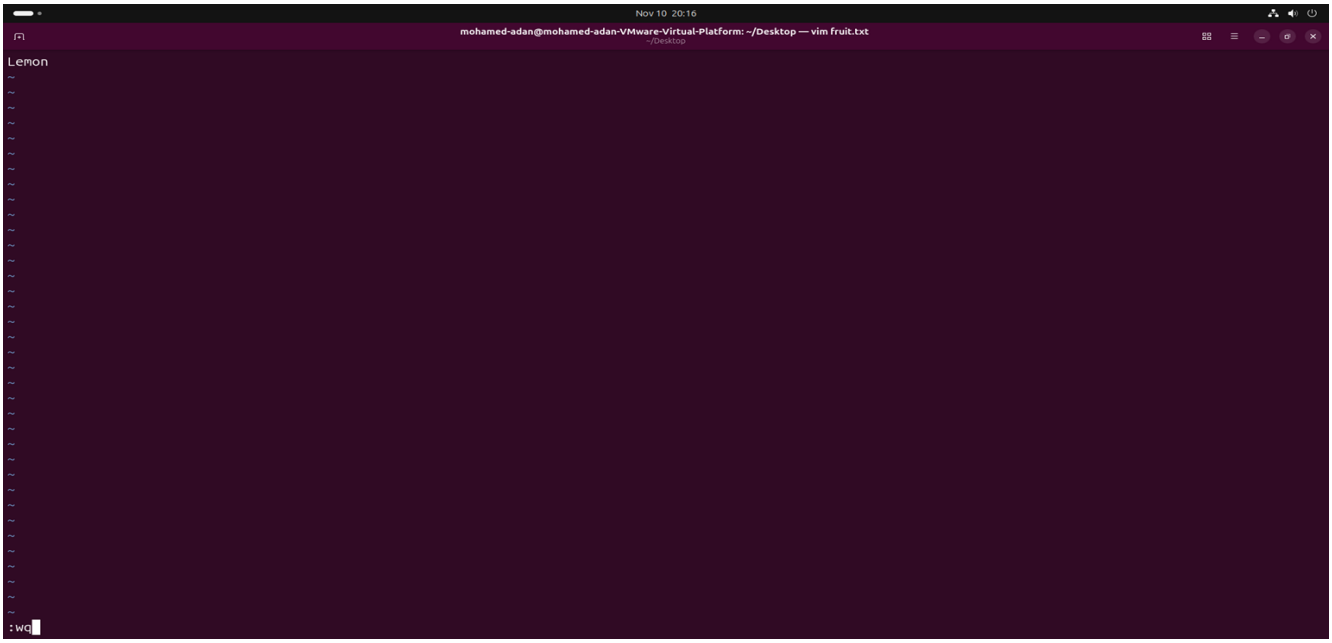
54. Text Editors: nano and vim

Linux provides **command-line text editors**.

1. **nano:** A simple, beginner-friendly editor that shows keyboard shortcuts at the bottom of the screen. Write nano [FileName] at the terminal, type something, then press Ctrl+O and Enter to save, and Ctrl+X to exit.



2. **vim:** A highly advanced, powerful editor. It uses different modes (Insert mode for typing, Normal mode for commands). Write vim [FileName] at the terminal, Press i to enter Insert mode, type something, press Esc, then type :wq and press Enter to save and quit.



55. Hard Links vs. Symbolic (Soft) Links

Linux allows you to create **shortcuts** to files.

1. **Hard Link:** Creates another name for the exact same file data on the hard drive. If you delete the original file, the hard link still works. You cannot create a hard link to a directory.
2. **Symbolic (Soft) Link:** Creates a shortcut that points to the original file's path (like a Windows shortcut). If you delete the original file, the symbolic link breaks (it becomes a "dead link"). You can create symbolic links to directories.

Options:

- **ln [original] [link]:** Creates a hard link.
- **ln -s [original] [link]:** Creates a symbolic (soft) link.



56. Processes, PIDs, and PPIDs

Every time you run a command, the kernel loads it into memory and gives it a unique number called a **PID** (Process ID). The process that started it is called the parent, and its number is the **PPID** (Parent Process ID). The very first process started by the kernel is **systemd**, which always has PID 1.



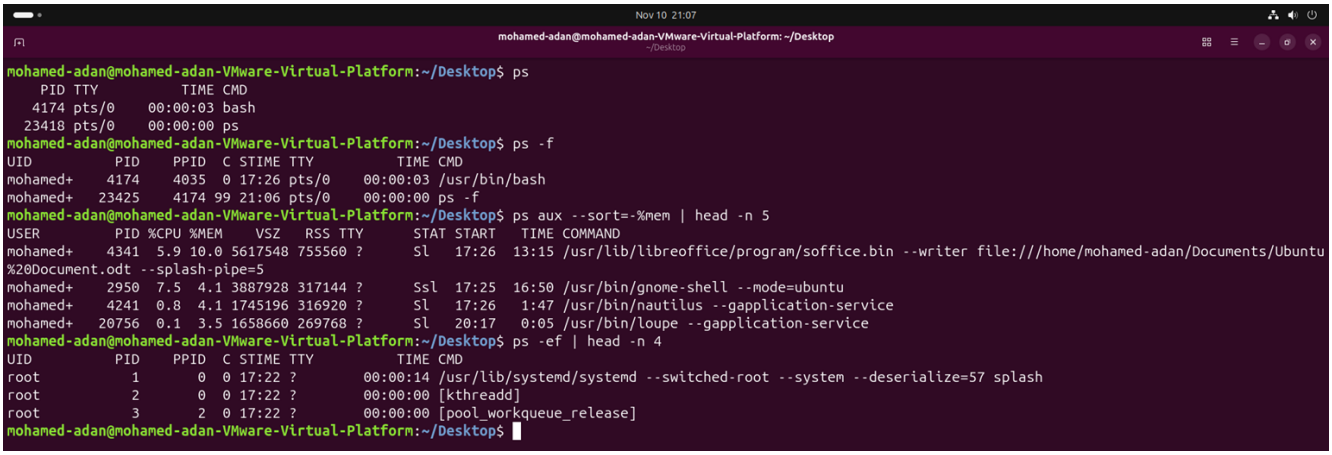
Note: echo $$ shows the PID of the shell you are currently typing in

57. The ps Command (Snapshots)

ps lists the processes that exist at the exact moment you press Enter. It does not update live.

Options:

- **ps:** Shows only the processes attached to your terminal.
- **ps -f:** (Full) Adds the owner, PPID, and start time.
- **ps -p [PID] -o [columns]:** Shows only a specific PID with chosen columns (e.g., -o pid,user,comm).
- **ps aux:** Shows every process from every user, including CPU and memory usage. (Often piped to less to read).
- **ps aux --sort=-%mem | head -n 5:** Sorts processes by memory usage (the minus sign means largest first).
- **ps -ef:** Another full format that clearly shows the PPID column.

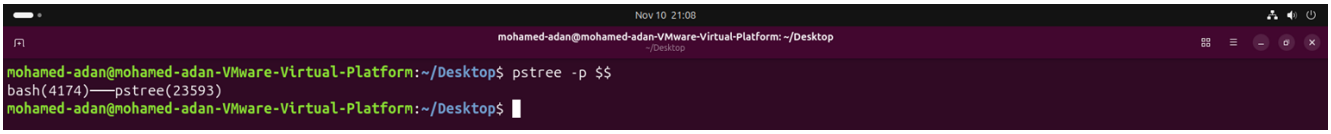


58. The pstree Command

Draws the **parent-child relationship** of processes as a tree, making it easy to see what started what.

Options:

- **-p:** Shows the PIDs in brackets next to the names.

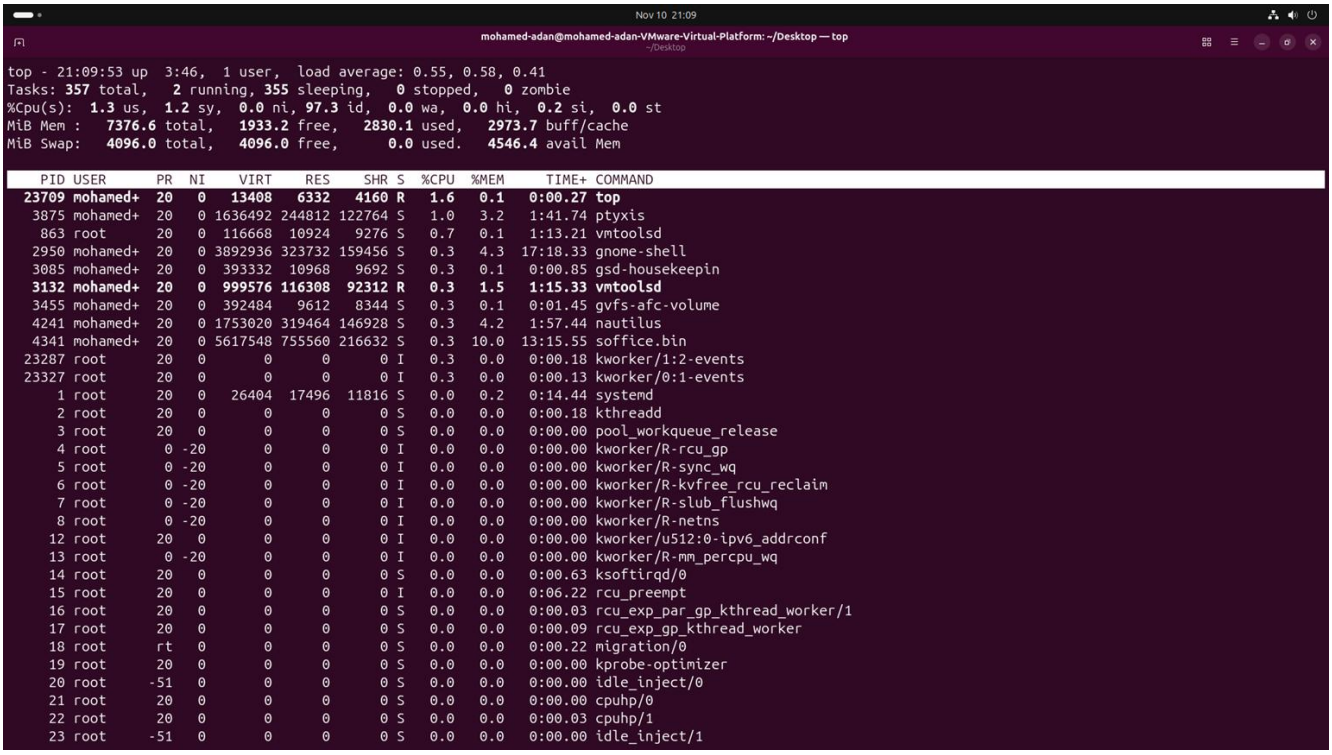


59. The top and htop Commands

top provides a live, redrawing view of processes, busiest first. **htop** is a more colorful, user-friendly alternative (may need to be installed).

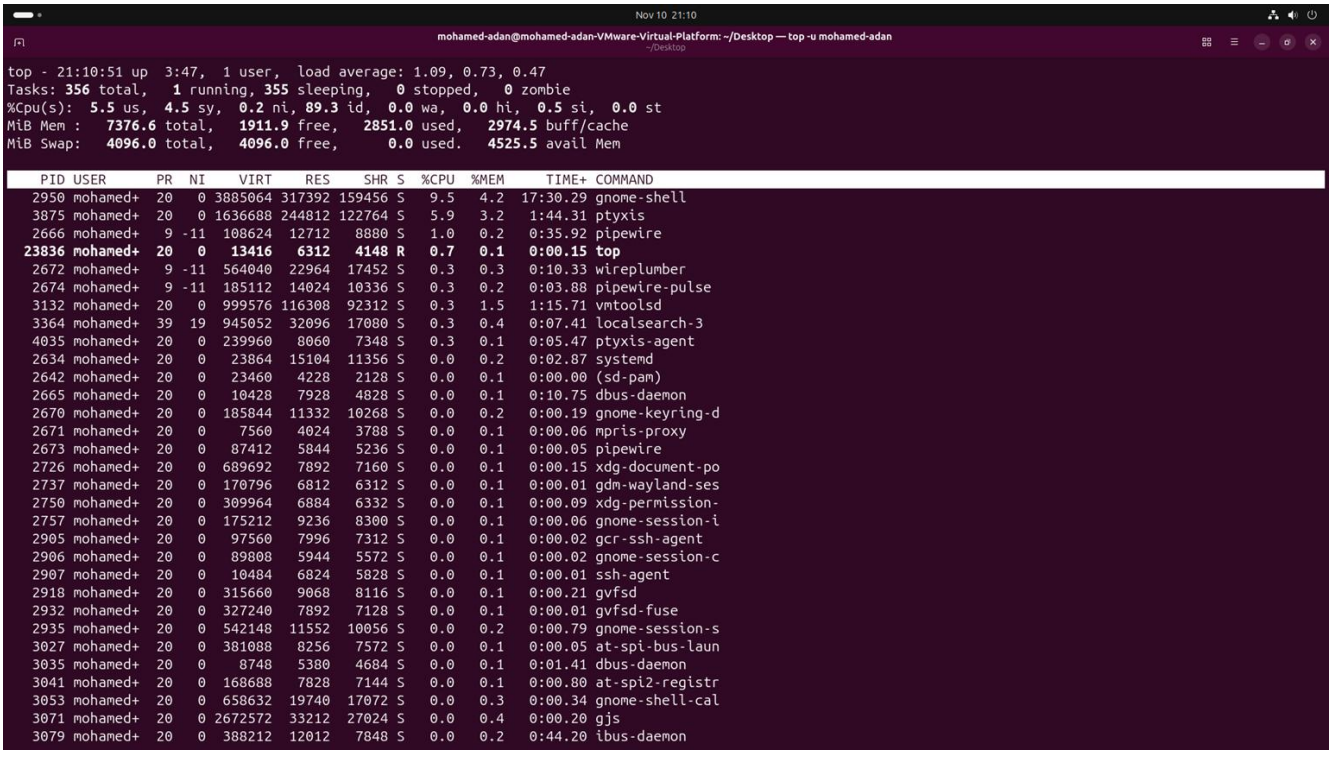
Options:

- **top:** -u [user] (show only that user's processes). Keys: P (sort by CPU), M (sort by memory), k (kill), r (renice), q (quit).
- **htop:** -u [user], -t (tree view). Keys: F6 (sort column), F9 (send signal), F10/q (quit).



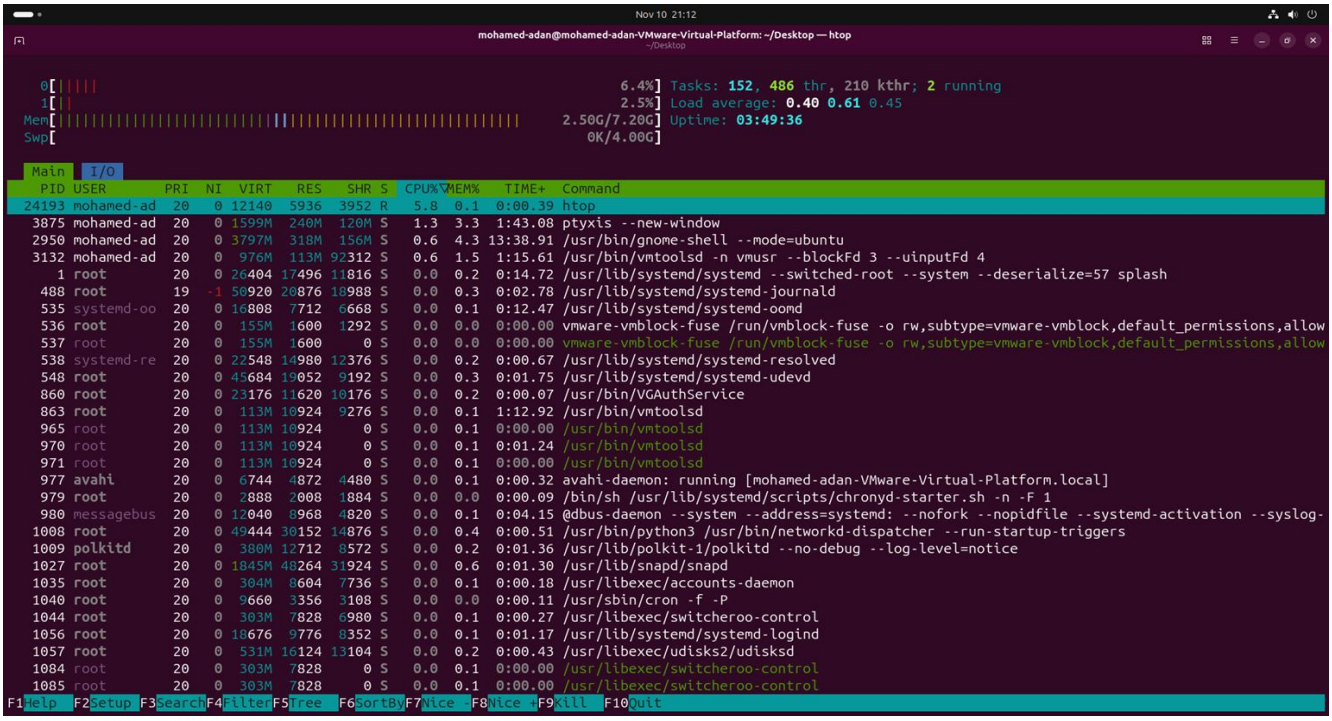

```
top - 21:09:53 up  3:46,  1 user,  load average: 0.55, 0.58, 0.41
Tasks: 357 total,   2 running, 355 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.3 us,  1.2 sy,  0.0 ni, 97.3 id,  0.0 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem :   7376.6 total,   1933.2 free,   2830.1 used,   2973.7 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   4546.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  23709 mohamed+  20   0   13408   6332   4160 R   1.6   0.1   0:00.27 top
   3875 mohamed+  20   0 1636492 244812 122764 S   1.0   3.2   1:41.74 ptyxis
    863 root      20   0  116668  10924   9276 S   0.7   0.1   1:13.21 vmtoolsd
   2950 mohamed+  20   0 3892936 323732 159456 S   0.3   4.3  17:18.33 gnome-shell
   3085 mohamed+  20   0  393332  10968   9692 S   0.3   0.1   0:00.85 gsd-housekeepin
   3132 mohamed+  20   0  999576 116308  92312 R   0.3   1.5   1:15.33 vmtoolsd
   3455 mohamed+  20   0  392484   9612   8344 S   0.3   0.1   0:01.45 gvfs-afc-volume
   4241 mohamed+  20   0 1753020 319464 146928 S   0.3   4.2   1:57.44 nautilus
   4341 mohamed+  20   0 5617548 755560 216632 S   0.3  10.0  13:15.55 soffice.bin
  23287 root      20   0       0      0      0 I   0.3   0.0   0:00.18 kworker/1:2-events
  23327 root      20   0       0      0      0 I   0.3   0.0   0:00.13 kworker/0:1-events
      1 root      20   0   26404  17496  11816 S   0.0   0.2   0:14.44 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.18 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.63 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:06.22 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.03 rcu_exp_par_gp_kthread_worker/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.09 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.22 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kprobe-optimizer
     20 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.03 cpuhp/1
     23 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
```




```
top - 21:10:51 up  3:47,  1 user,  load average: 1.09, 0.73, 0.47
Tasks: 356 total,   1 running, 355 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.5 us,  4.5 sy,  0.2 ni, 89.3 id,  0.0 wa,  0.0 hi,  0.5 si,  0.0 st
MiB Mem :   7376.6 total,   1911.9 free,   2851.0 used,   2974.5 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   4525.5 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2950 mohamed+  20   0 3885064 317392 159456 S   9.5   4.2  17:30.29 gnome-shell
   3875 mohamed+  20   0 1636688 244812 122764 S   5.9   3.2   1:44.31 ptyxis
   2666 mohamed+   9 -11  108624  12712   8880 S   1.0   0.2   0:35.92 pipewire
  23836 mohamed+  20   0   13416   6312   4148 R   0.7   0.1   0:00.15 top
   2672 mohamed+   9 -11  564040  22964  17452 S   0.3   0.3   0:10.33 wireplumber
   2674 mohamed+   9 -11  185112  14024  10336 S   0.3   0.2   0:03.88 pipewire-pulse
   3132 mohamed+  20   0  999576 116308  92312 S   0.3   1.5   1:15.71 vmtoolsd
   3364 mohamed+  39  19  945052  32096  17080 S   0.3   0.4   0:07.41 localsearch-3
   4035 mohamed+  20   0  239960   8060   7348 S   0.3   0.1   0:05.47 ptyxis-agent
   2634 mohamed+  20   0   23864  15104  11356 S   0.0   0.2   0:02.87 systemd
   2642 mohamed+  20   0   23460   4228   2128 S   0.0   0.1   0:00.00 (sd-pam)
   2665 mohamed+  20   0   10428   7928   4828 S   0.0   0.1   0:10.75 dbus-daemon
   2670 mohamed+  20   0  185844  11332  10268 S   0.0   0.2   0:00.19 gnome-keyring-d
   2671 mohamed+  20   0    7560   4024   3788 S   0.0   0.1   0:00.06 mpris-proxy
   2673 mohamed+  20   0   87412   5844   5236 S   0.0   0.1   0:00.05 pipewire
   2726 mohamed+  20   0  689692   7892   7160 S   0.0   0.1   0:00.15 xdg-document-po
   2737 mohamed+  20   0  170796   6812   6312 S   0.0   0.1   0:00.01 gdm-wayland-ses
   2750 mohamed+  20   0  309964   6884   6332 S   0.0   0.1   0:00.09 xdg-permission-
   2757 mohamed+  20   0  175212   9236   8300 S   0.0   0.1   0:00.06 gnome-session-i
   2905 mohamed+  20   0   97560   7996   7312 S   0.0   0.1   0:00.02 gcr-ssh-agent
   2906 mohamed+  20   0   89808   5944   5572 S   0.0   0.1   0:00.02 gnome-session-c
   2907 mohamed+  20   0   10484   6824   5828 S   0.0   0.1   0:00.01 ssh-agent
   2918 mohamed+  20   0  315660   9068   8116 S   0.0   0.1   0:00.21 gvfsd
   2932 mohamed+  20   0  327240   7892   7128 S   0.0   0.1   0:00.01 gvfsd-fuse
   2935 mohamed+  20   0  542148  11552  10056 S   0.0   0.2   0:00.79 gnome-session-s
   3027 mohamed+  20   0  381088   8256   7572 S   0.0   0.1   0:00.05 at-spi-bus-laun
   3035 mohamed+  20   0    8748   5380   4684 S   0.0   0.1   0:01.41 dbus-daemon
   3041 mohamed+  20   0  168688   7828   7144 S   0.0   0.1   0:00.80 at-spi2-registr
   3053 mohamed+  20   0  658632  19740  17072 S   0.0   0.3   0:00.34 gnome-shell-cal
   3071 mohamed+  20   0 2672572  33212  27024 S   0.0   0.4   0:00.20 gjs
   3079 mohamed+  20   0  388212  12012   7848 S   0.0   0.2   0:44.20 ibus-daemon
```



60. Foreground, Background, and Job Control

A **foreground job** holds your terminal until it ends. Adding & runs it in the background, giving your prompt back immediately.

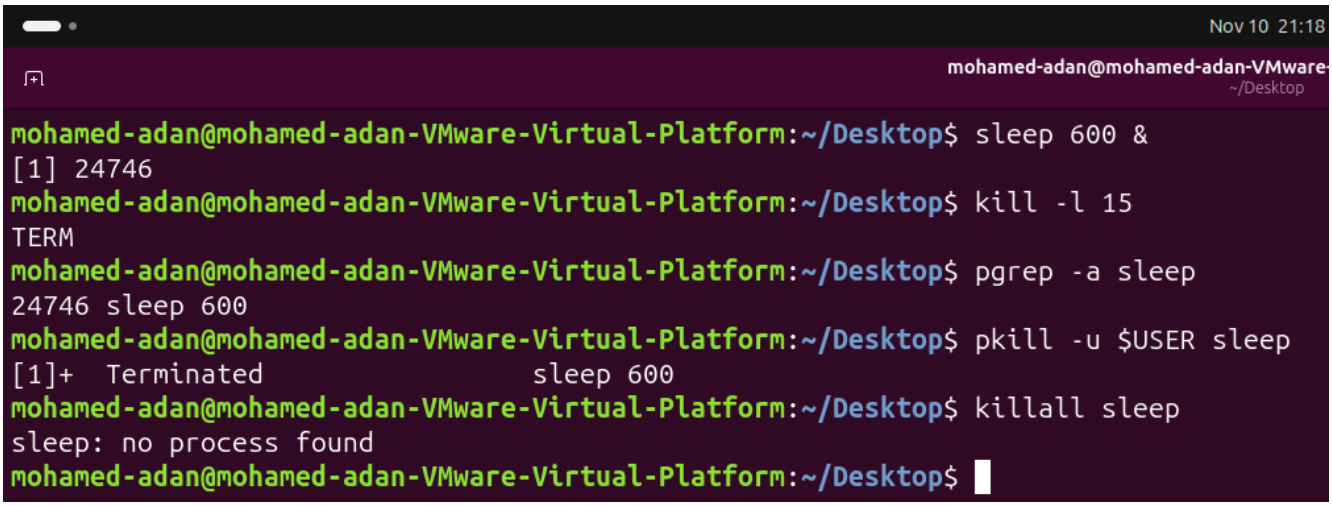
- **Ctrl+C (SIGINT):** Kills the foreground job.
- **Ctrl+Z (SIGTSTP):** Pauses the foreground job and puts it in the background.
- **jobs -l:** Lists background jobs and their PIDs.
- **bg %N:** Resumes a stopped job in the background.
- **fg %N:** Brings a background/stopped job back to the foreground.



61. Signals, kill, killall, pkill

Signals tell processes what to do.

- **kill -l:** Lists all signal names.
- **kill -l 15:** Turns a number into its name (TERM).
- **kill -HUP [PID]:** (SIGHUP) Tells services to reload.
- **kill [PID]:** Sends SIGTERM (15), a polite request to exit.
- **kill -9 [PID]:** Sends SIGKILL (9), forcing it to end immediately (last resort).
- **killall [name]:** Kills by exact name.
- **pkill -u [user] [name]:** Kills by name, limited to a specific user.
- **pgrep -a [name]:** Shows the PID and command of matching processes before you kill them.



62. Niceness and Priority (nice, renice)

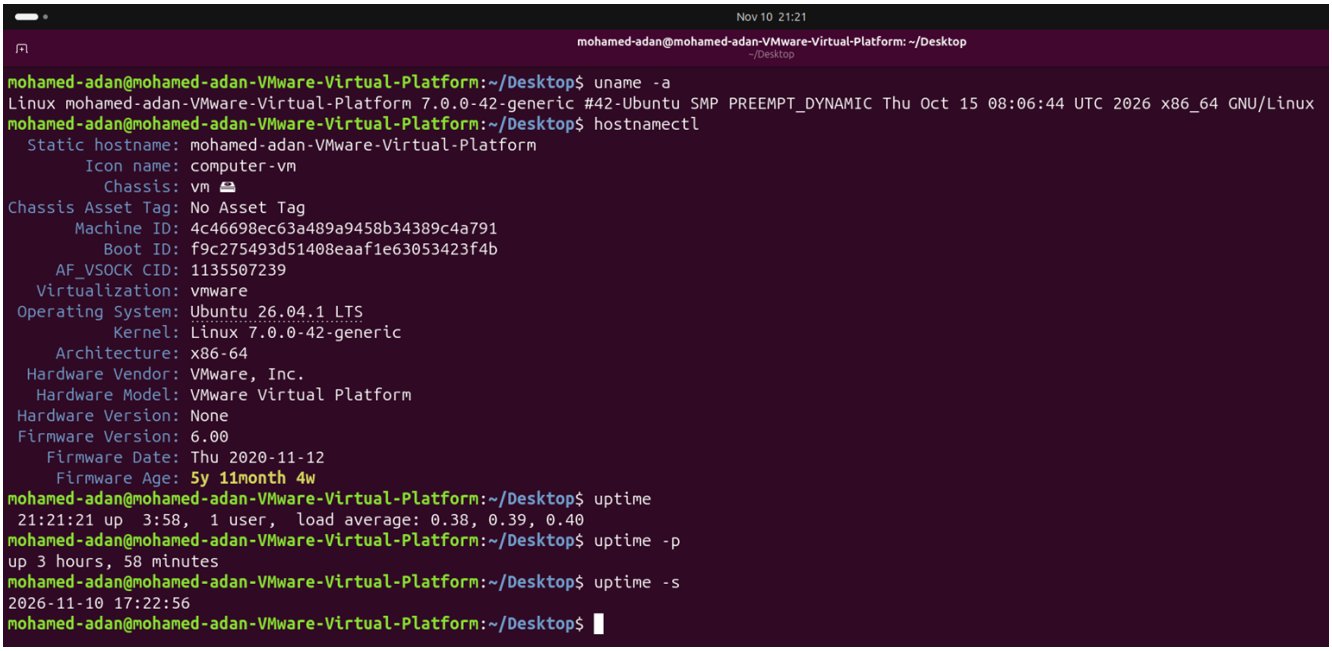
Niceness ranges from -20 (greediest, needs sudo) to 19 (kindest). Default is 0.

- **nice -n 10 sleep 300 &:** Starts a command with niceness 10.
- **ps -o pid,ni,comm -p [PID]:** Checks the NI column.
- **renice -n 15 -p [PID]:** Changes priority of a running process (normal users can only increase the number).



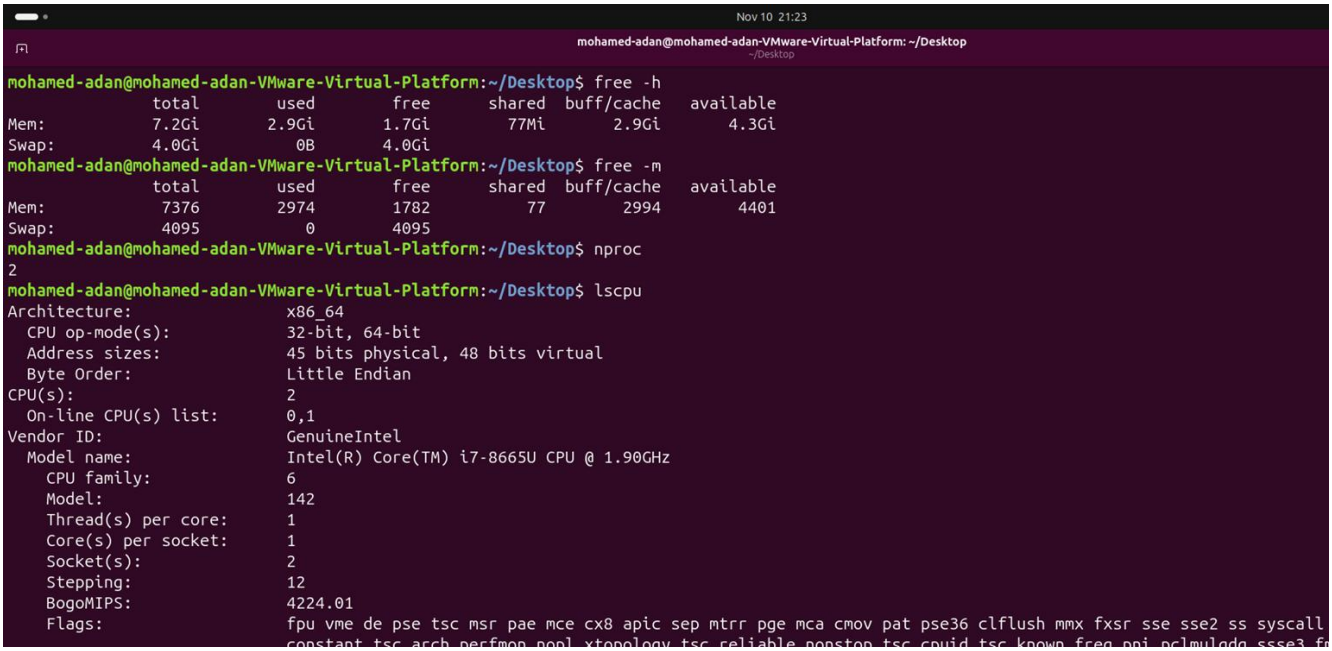
63. System Information (uname, hostnamectl, uptime)

- **uname:** -r (kernel release), -m (machine type), -n (hostname), -a (all).
- **hostnamectl:** Shows hostname, OS, kernel, and virtualization (e.g., Oracle/VirtualBox).
- **uptime:** Shows time, users, and load average (1, 5, 15 mins).
- **uptime -p:** Time since boot in words.
- **uptime -s:** Exact boot time.



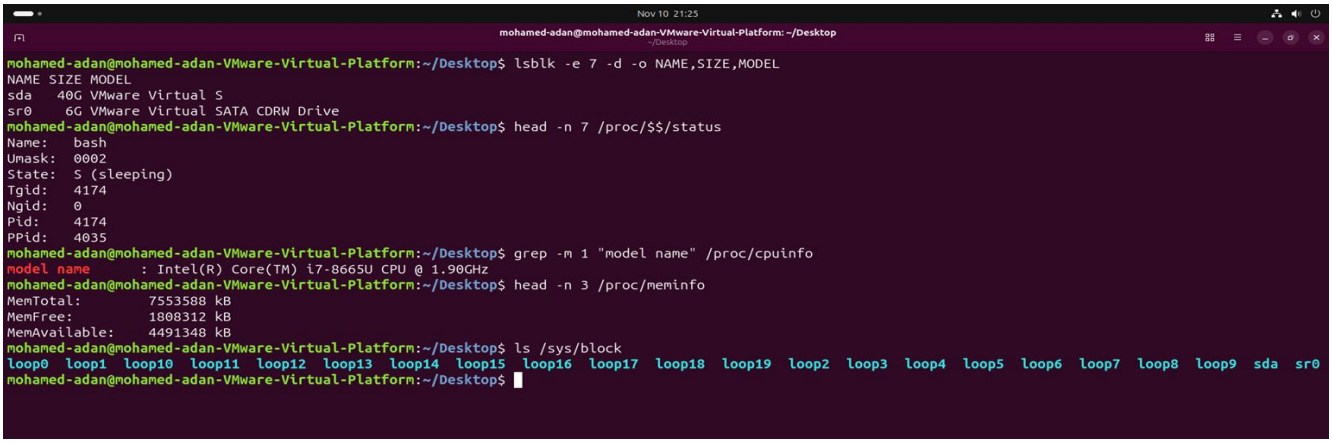
64. Memory and CPU (free, lscpu, nproc)

- **free -h:** Shows RAM/Swap in human-readable units. (Read the "available" column).
- **free -m:** Shows in MiB.
- **free -h -s 2:** Updates the table every 2 seconds.
- **lscpu:** Summarizes CPU architecture.
- **nproc:** Shows the number of CPUs (compare to load average).



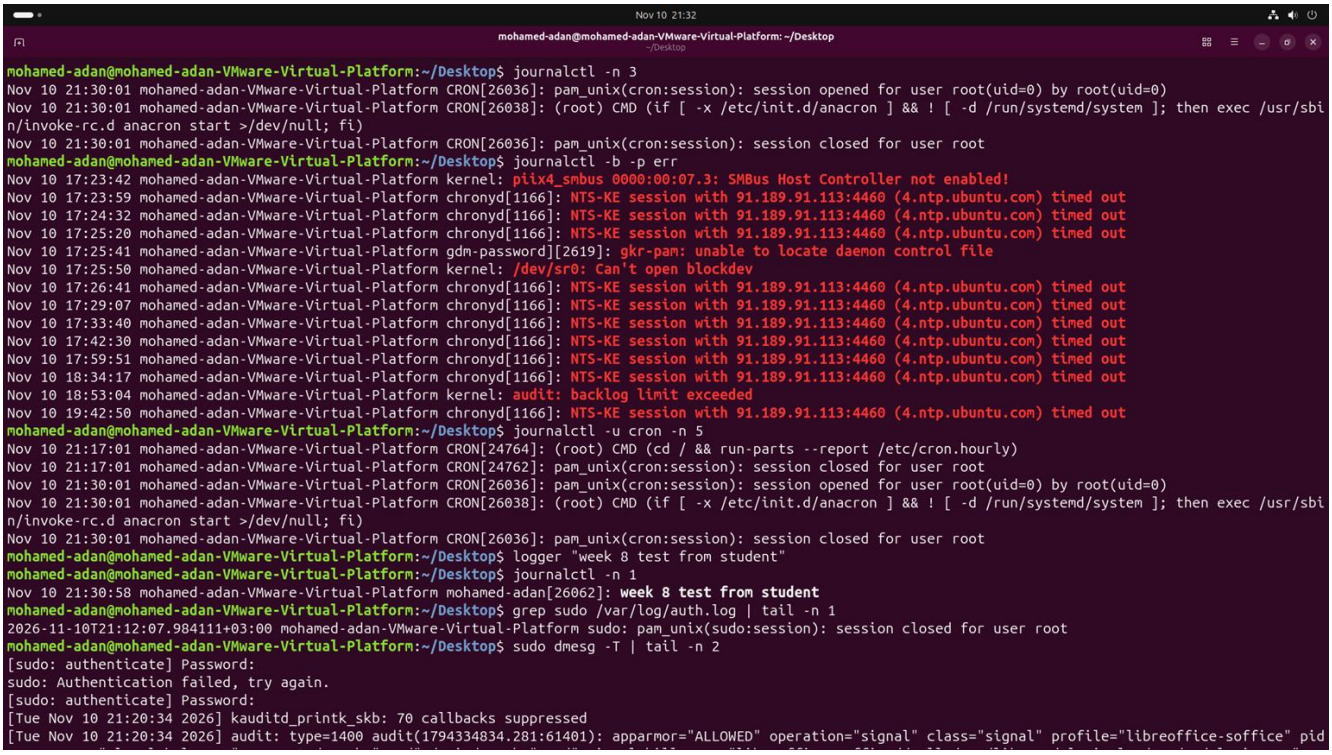
65. Disks and Virtual Directories (lsblk, /proc, /sys)

- **lsblk:** Lists block devices.
- **-e 7:** Hides loop devices (snaps).
- **-d -o NAME,SIZE,MODEL:** Shows whole disks only, with the model.
- **/proc:** Kernel's live window (files show size 0 but contain text).
- **cat /proc/uptime, /proc/loadavg.**
- **/proc/[PID]/status:** Shows name, state, PID, PPID.
- **/proc/[PID]/comm:** Shows just the command name.
- **/sys:** Devices as files.
- **ls /sys/block:** Shows disks.
- **cat /sys/class/net/enp0s3/address:** Shows MAC address.



66. Logs and Troubleshooting (journalctl, /var/log, dmesg)

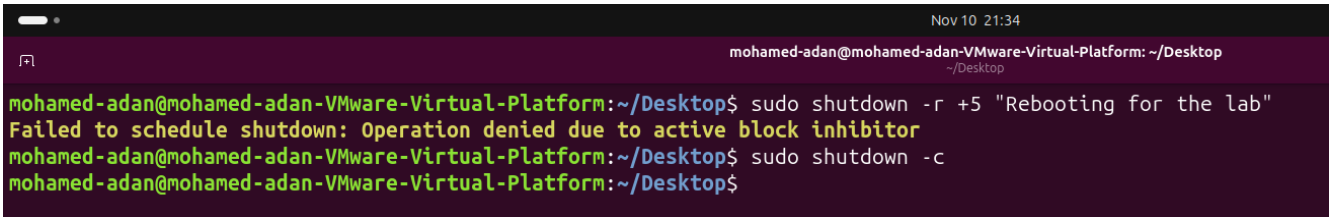
- **journalctl:** Reads the systemd journal.
- **-n N:** Last N lines.
- **-b:** Current boot only.
- **-p err:** Errors only.
- **-u [unit]:** Specific service.
- **-xe:** Jumps to end with explanations.
- **-f:** Follow live logs.
- **/var/log:** Classic text logs (syslog, auth.log, kern.log).
- **grep sudo /var/log/auth.log:** Shows sudo usage.
- **sudo dmesg -T:** Kernel messages with human-readable timestamps. (Normal user gets "Operation not permitted").
- **logger:** Writes a test line to the system log.



67. Shutdown and Reboot

Always **shut down cleanly** from inside Linux.

- **sudo shutdown -r +5 "Rebooting for the lab":** Schedules a reboot in 5 minutes.
- **sudo shutdown -c:** Cancels the scheduled shutdown.
- **sudo reboot:** Restarts now.
- **sudo poweroff:** Shuts down now.



68. The Troubleshooting Orders

1. **If a service misbehaves:**

- **systemctl status [service]** (Is it running? What did the last log say?)
- **journalctl -u [service]** (Find the first error).
- **Check /etc/[service]/** (What config changed recently?).

2. **If the machine is slow:**

- **uptime** (Check load average against nproc).
- **free -h** (Is memory short? Is swap in use?).
- **top** (Press M to sort by memory, note the PID at the top, then decide to wait, renice, or kill).